

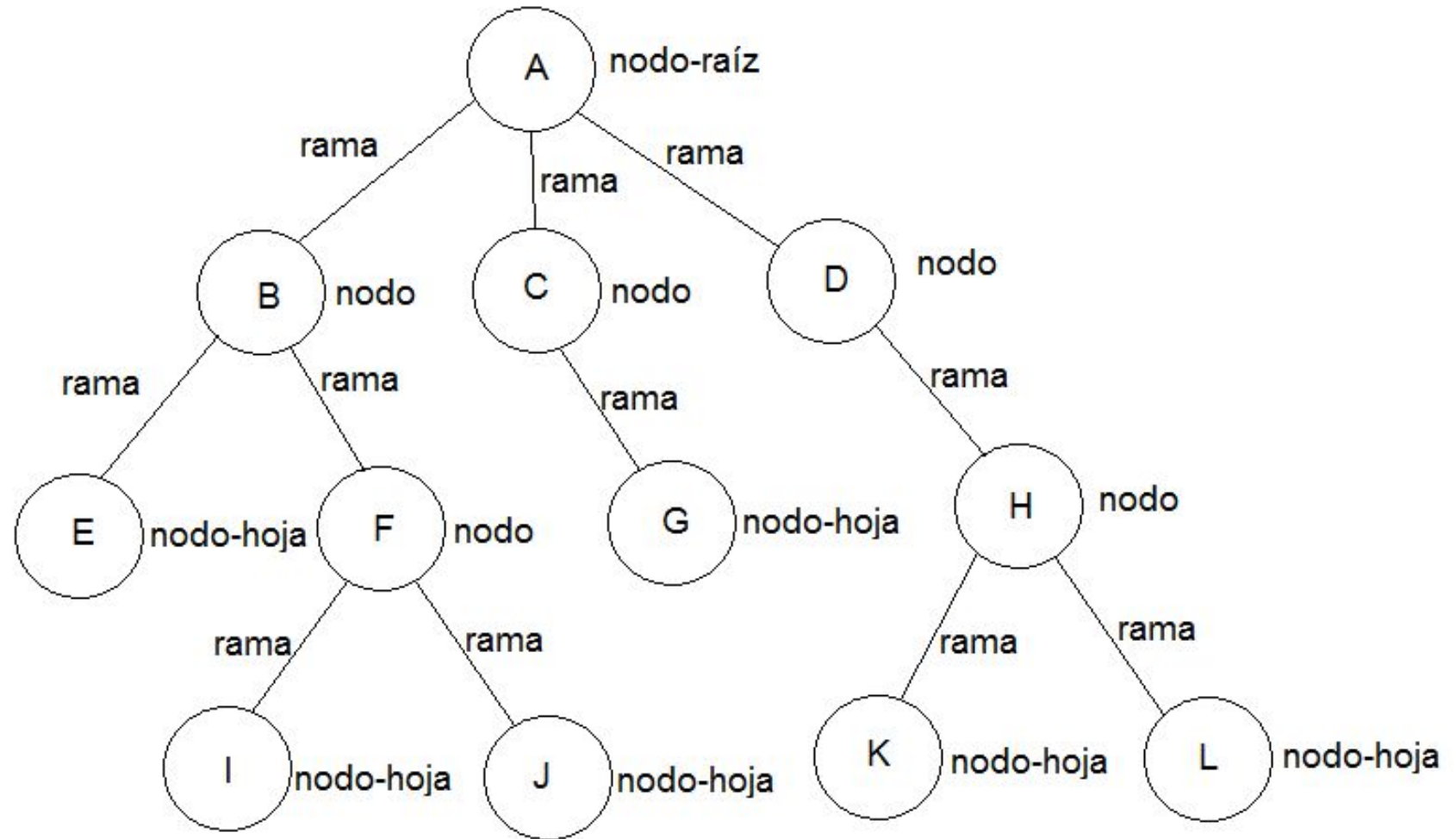


FUNDAMENTOS Y TEORÍA DE LA COMPUTACIÓN

ÁRBOLES Y GRAFOS

ÁRBOLES:

Un árbol es una estructura de datos no lineal formada por nodos y en la que cada nodo puede apuntar a uno o varios nodos a través de arcos o ramas.



ÁRBOLES Y GRAFOS

Raíz y hojas:

El nodo superior del árbol es la raíz (en nuestro ejemplo el nodo etiquetado con la letra A) que puede tener uno o más nodos hijos o descendientes.

La raíz es un nodo especial que no tiene padres.

Un nodo hoja es otro tipo especial de nodos que no tiene hijos (son nodos terminales). En nuestro ejemplo, los nodos hojas son: E, I, J, G, K y L.

ÁRBOLES Y GRAFOS

Camino, altura, grado y profundidad:

Se define camino de un árbol como cualquier secuencia de nodos que cumple que cada nodo es padre del siguiente en la secuencia. La longitud del camino se define como el número de nodos menos uno.

Se define altura de un nodo como la longitud del camino más largo que comienza en el nodo y termina en una hoja. La altura de una hoja es cero. La altura de un árbol es la altura de la raíz.

El grado de un nodo es el número de hijos descendientes.

La profundidad de un nodo es la longitud del camino que comienza en la raíz y termina en el nodo. La profundidad de un nodo se conoce como nivel del nodo.

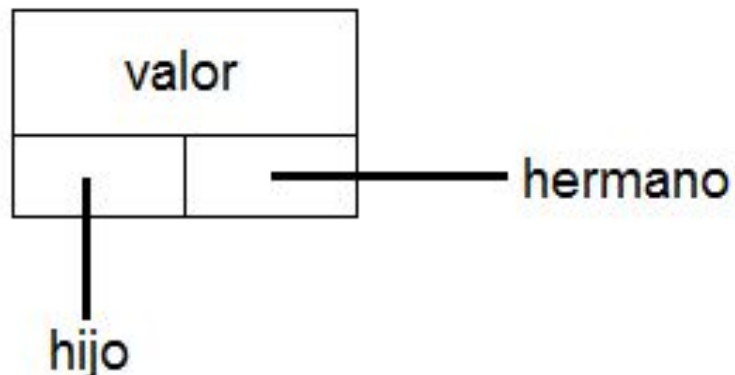
ÁRBOLES Y GRAFOS

Recorrido de un árbol:

El recorrido de un árbol es el proceso de visitar de manera sistemática exactamente una vez cada nodo.

Representación de árboles:

Una forma de representar árboles generales es con una estructura que tiene el valor del nodo (etiqueta) y dos punteros, uno al primer hijo, y otro al hermano más cercano:

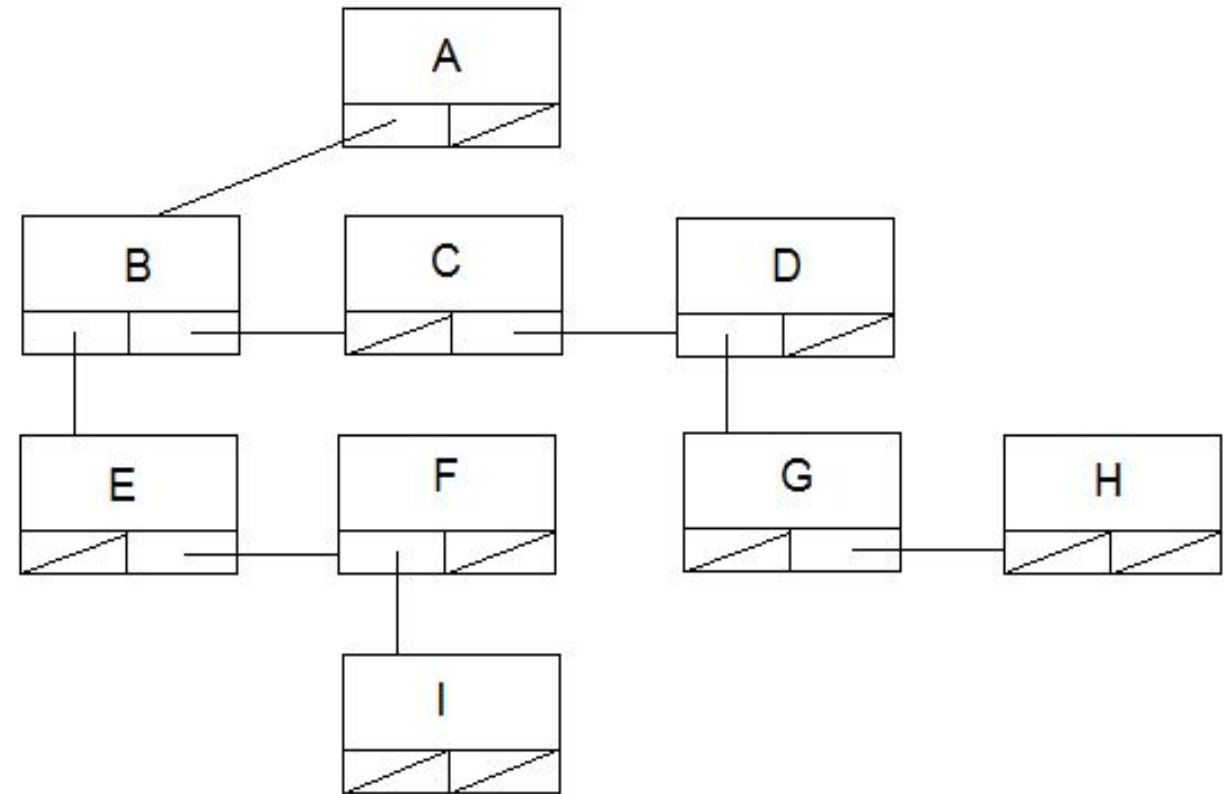
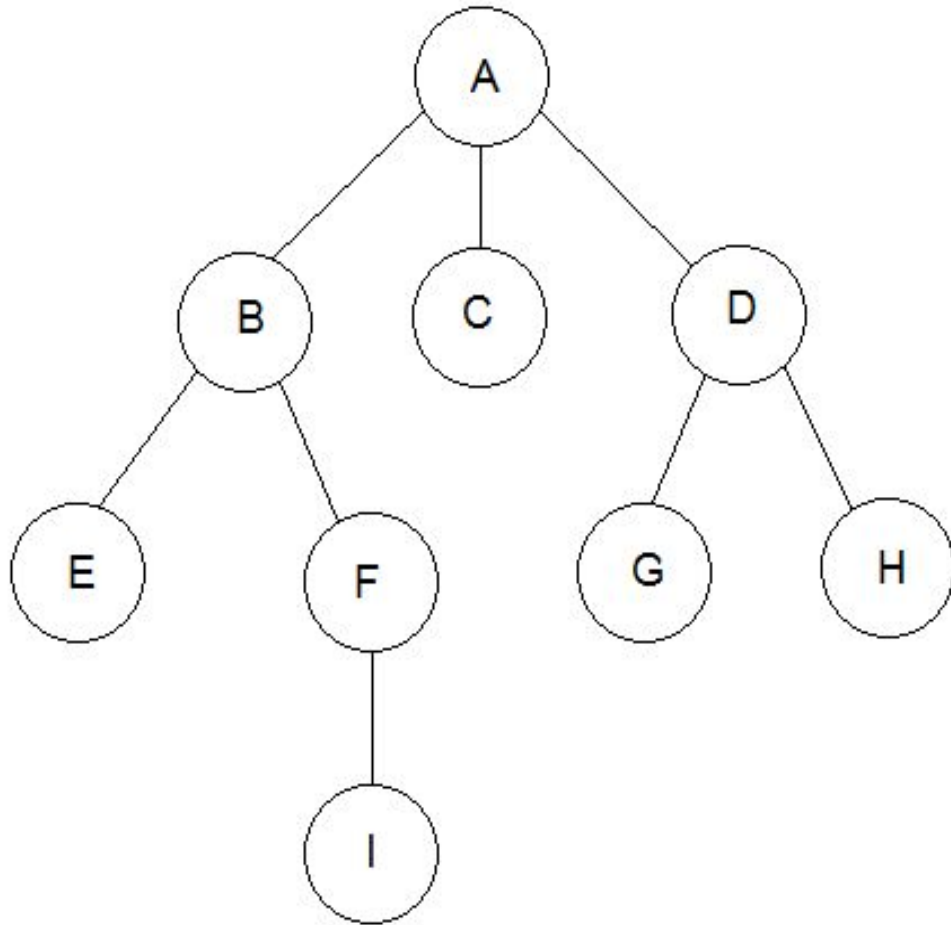


```
typedef struct tipo_nodo{  
    int valor;  
    struct tipo_nodo *hijo;  
    struct tipo_nodo *hno;  
}nodo;
```

```
class Arbol(  
    int valor;  
    class Arbol hijo;  
    class Arbol hno;  
);
```

ÁRBOLES Y GRAFOS

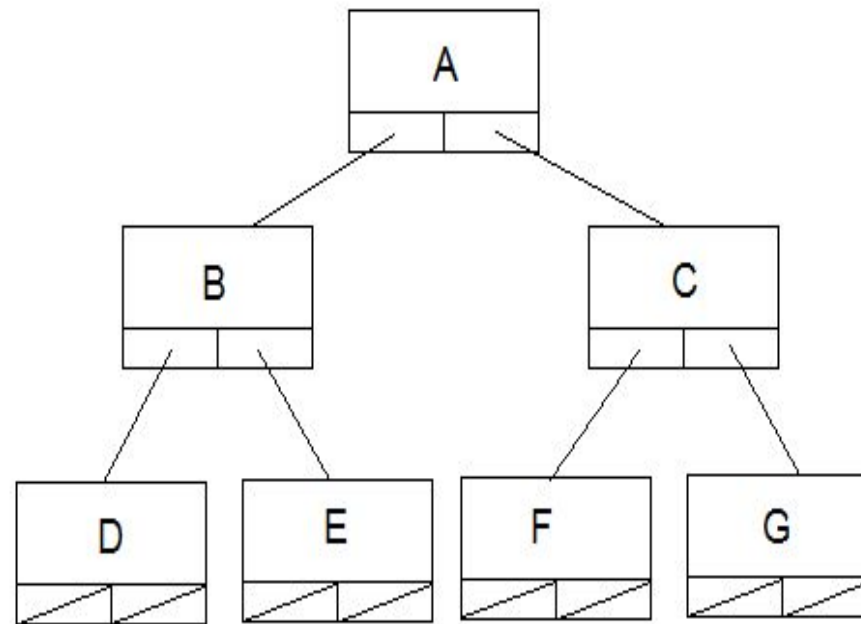
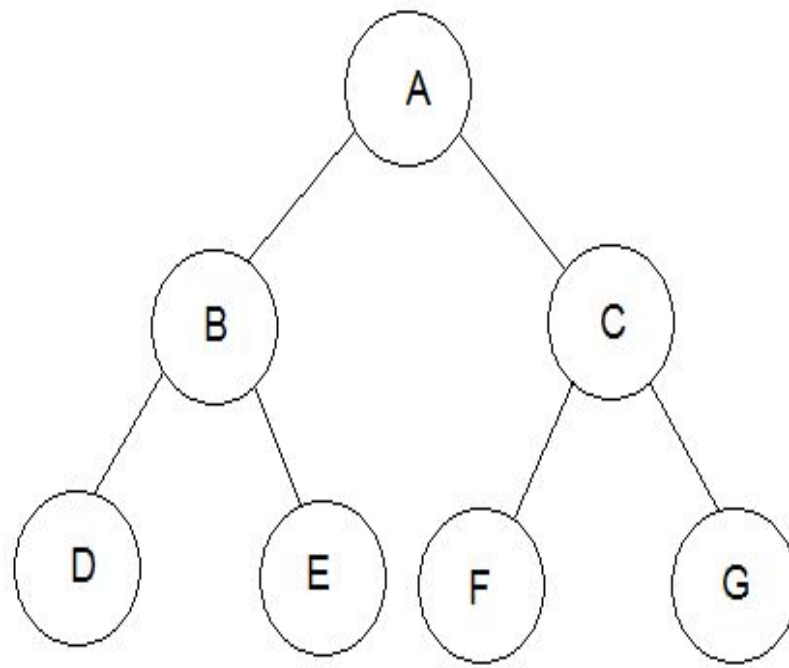
Representación de árboles (cont.):



ÁRBOLES Y GRAFOS

Árboles Binarios:

Los árboles binarios son un tipo particular de árboles muy utilizados por su agilidad en la búsqueda de datos. La particularidad que tiene es que cada nodo puede tener a lo sumo dos hijos, uno izquierdo y otro derecho:



```
typedef struct tipo_nodo{  
    int valor;  
    struct tipo_nodo *izq;  
    struct tipo_nodo *der;  
}nodo;
```

```
class ArbolBinario(  
    int valor;  
    class ArbolBinario izq;  
    class ArbolBinario der;  
);
```

Árbol Binario de Búsqueda:

Los árboles binarios de búsqueda son aquellos en los que existe cierto orden, como puede ser un ordenamiento en el que todos los elementos del subárbol izquierdo de un nodo cualquiera tienen valores menores que ese nodo y los de la derecha valores mayores.

Los recorridos pueden ser en profundidad o anchura. A su vez, el recorrido en profundidad puede ser: pre-orden, in-orden o pos-orden.

ÁRBOLES Y GRAFOS

Recorridos de un árbol binario:

Pre-orden: primero se visita la raíz, luego el subárbol izquierdo y luego el subárbol derecho.

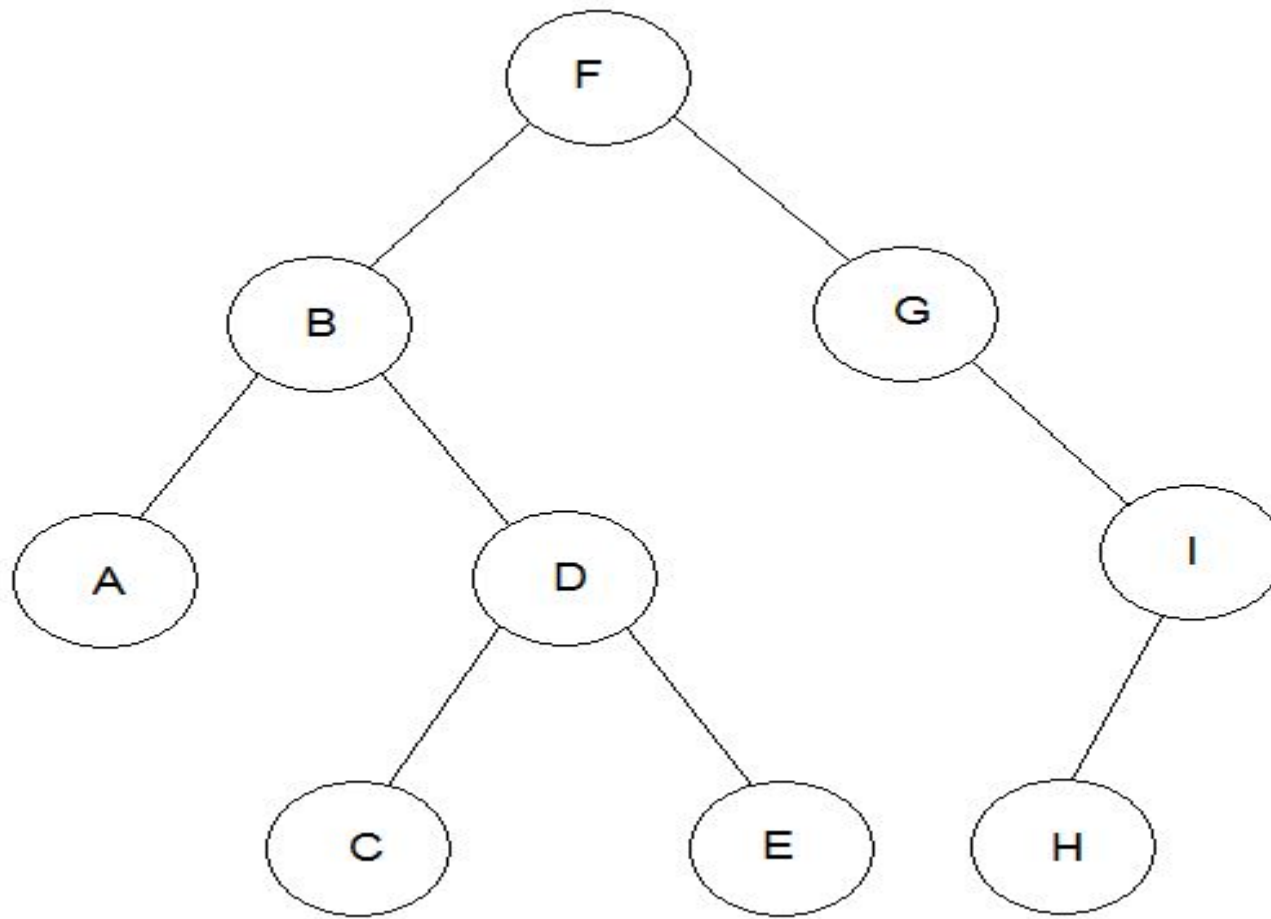
In-orden: primero se visita el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho.

Pos-orden: primero se recorre el subárbol izquierdo, luego el subárbol derecho y finalmente la raíz.

Anchura (por nivel): se va recorriendo el árbol por niveles comenzando por la raíz.

ÁRBOLES Y GRAFOS

Ejemplo de recorridos:



PRE-ORDEN:
F-B-A-D-C-E-G-I-H
(RAIZ-IZQUIERDA-DERECHA)

IN-ORDEN:
A-B-C-D-E-F-G-H-I
(IZQUIERDA-RAIZ-DERECHA)

POS-ORDEN:
A-C-E-D-B-H-I-G-F
(IZQUIERDA-DERECHA-RAIZ)

ANCHURA:
F-B-G-A-D-I-C-E-H
(POR NIVELES)

ÁRBOLES Y GRAFOS

Implementación de Árboles Binarios de Búsqueda:

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  typedef struct tipo_nodo{
4      int valor;
5      struct tipo_nodo *izq;
6      struct tipo_nodo *der;
7  }nodo;
8  typedef struct tipo_nodoCola{
9      nodo *dato;
10     struct tipo_nodoCola *sig;
11 }nodoCola;
12 typedef struct tipoCola{
13     nodoCola *primero;
14     nodoCola *ultimo;
15 }tCola;
16 /*Prototipo de funciones*/
17 /*Arbol*/
18 void crear_arbol(nodo *arbol, int dato);
19 void insertar_nodo(nodo *arbol, int dato);
20 void preorden(nodo arbol);
21 void inorden(nodo arbol);
22 void posorden(nodo arbol);
23 void anchura(nodo arbol);
24 /*Cola*/
25 void crearCola(tCola *cola);
26 void encolar(tCola *cola, nodo *dato);
27 nodo *desencolar(tCola *cola);
28 int vacia(tCola cola);
29 /*Main*/
30 main(){
31     nodo arbol;
32     int dato;
33     printf("Ingresar el primer elemento del arbol \n");
34     scanf("%d", &dato);
35     crear_arbol(&arbol, dato);
36     /*Cuerpo principal del programa*/
37 }
```

```
38 /*Definición de funciones*/
39 void crear_arbol(nodo *arbol, int dato){
40     arbol->izq=NULL;
41     arbol->der=NULL;
42     arbol->valor=dato;
43 }
44 void insertar_nodo(nodo *arbol, int dato){
45     nodo *nuevo=(nodo *)malloc(sizeof(nodo));
46     nuevo->valor=dato;
47     nuevo->izq=NULL;
48     nuevo->der=NULL;
49     nodo *actual, *anterior;
50     actual=arbol;
51     anterior=NULL;
52     while(actual!=NULL){
53         anterior=actual;
54         if(actual->valor>dato)
55             actual=actual->izq;
56         else
57             actual=actual->der;
58     }
59     if(anterior->valor>dato)
60         anterior->izq=nuevo;
61     else
62         anterior->der=nuevo;
63 }
64 void preorden(nodo arbol){
65     printf("%d-", arbol.valor);
66     if(arbol.izq!=NULL)
67         preorden(*arbol.izq);
68     if(arbol.der!=NULL)
69         preorden(*arbol.der);
70 }
```

```
71 void inorden(nodo arbol){
72     if(arbol.izq!=NULL)
73         inorden(*arbol.izq);
74     printf("%d-", arbol.valor);
75     if(arbol.der!=NULL)
76         inorden(*arbol.der);
77 }
78 void posorden(nodo arbol){
79     if(arbol.izq!=NULL)
80         posorden(*arbol.izq);
81     if(arbol.der!=NULL)
82         posorden(*arbol.der);
83     printf("%d-", arbol.valor);
84 }
85 void anchura(nodo arbol){
86     tCola cola;
87     crearCola(&cola);
88     nodo *aux;
89     aux=&arbol;
90     encolar(&cola, aux);
91     while(!vacia(cola)){
92         aux=desencolar(&cola);
93         printf("%d-", aux->valor);
94         if(aux->izq!=NULL)
95             encolar(&cola, aux->izq);
96         if(aux->der!=NULL)
97             encolar(&cola, aux->der);
98     }
99 }
100 void crearCola(tCola *cola){
101     /*Cuerpo de la función*/
102 }
103 void encolar(tCola *cola, nodo *dato){
104     /*Cuerpo de la función*/
105 }
106 nodo *desencolar(tCola *cola){
107     /*Cuerpo de la función*/
108 }
109 int vacia(tCola cola){
110     /*Cuerpo de la función*/
111 }
```

ÁRBOLES Y GRAFOS

Árboles AVL:

Los árboles AVL son árboles binarios de búsqueda equilibrados. Esto quiere decir que la altura de la rama izquierda no difiere en más de una unidad de la altura de la rama derecha.

Se define el factor de equilibrio Fe de un nodo como la diferencia entre la altura del subárbol izquierdo y la altura del subárbol derecho:

$$Fe = altura(arbol.izq) - altura(arbol.der)$$

El factor de equilibrio puede almacenarse en cada nodo o calcularse cuando se necesita.

ÁRBOLES Y GRAFOS

Arboles AVL (cont.):

En cualquier nodo de un árbol AVL debe cumplirse $|F_e| < 2$. Para conseguir esta propiedad de equilibrio, la inserción y borrado de los nodos se ha de realizar de una forma específica.

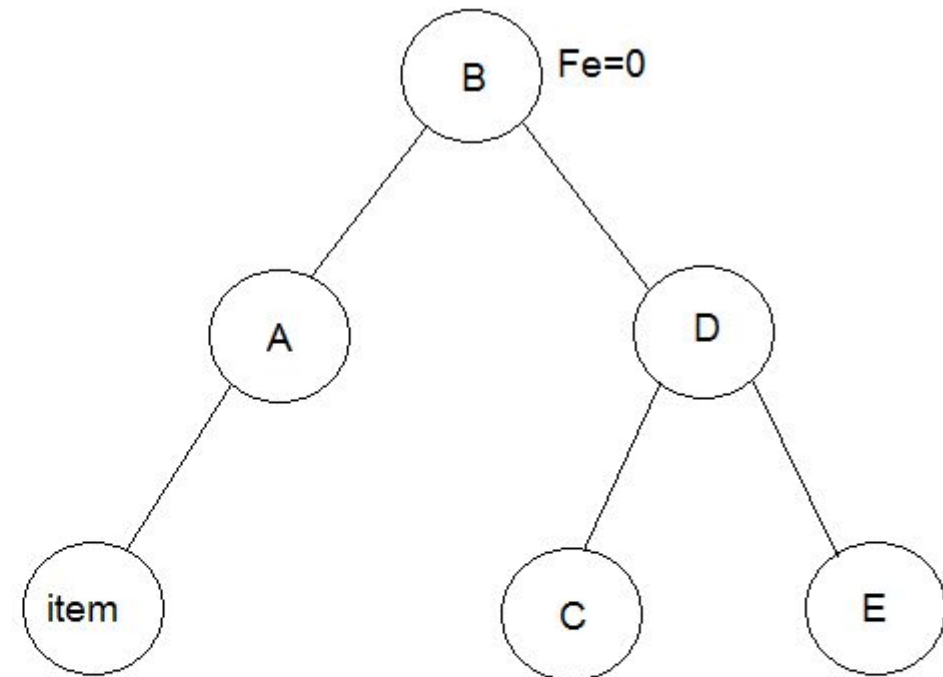
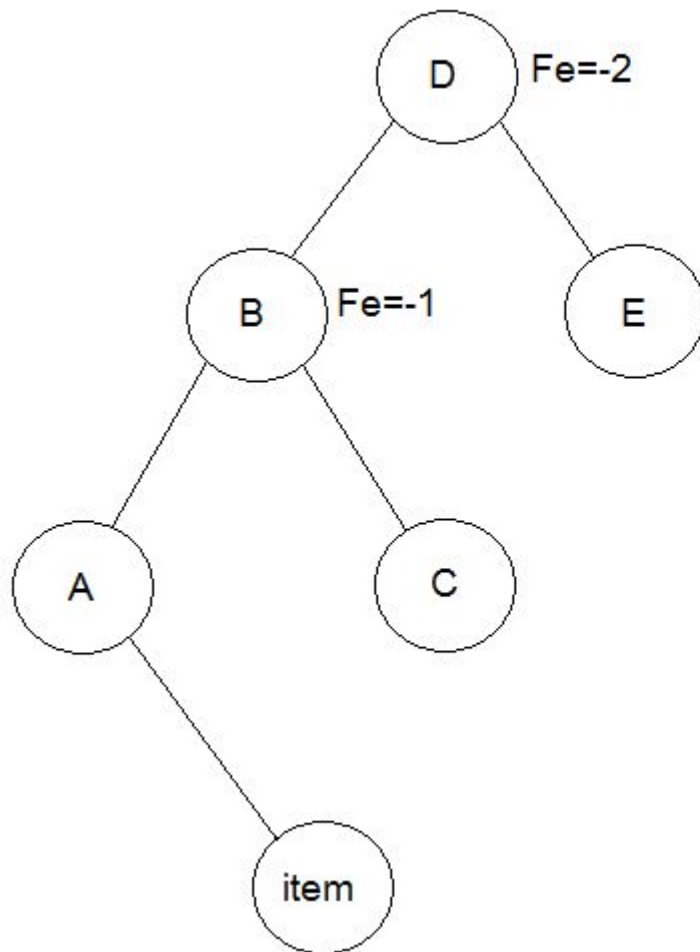
Si al insertar o borrar se rompe el equilibrio, se deben realizar una serie de rotaciones.

Las rotaciones pueden ser simples o dobles y a izquierda o derecha.

Cuando se inserta o borra un elemento, debe verificarse si se rompió el equilibrio. Para ello se van actualizando el F_e de cada nodo hacia arriba hasta llegar a la raíz, realizando las rotaciones necesarias para recuperar el equilibrio.

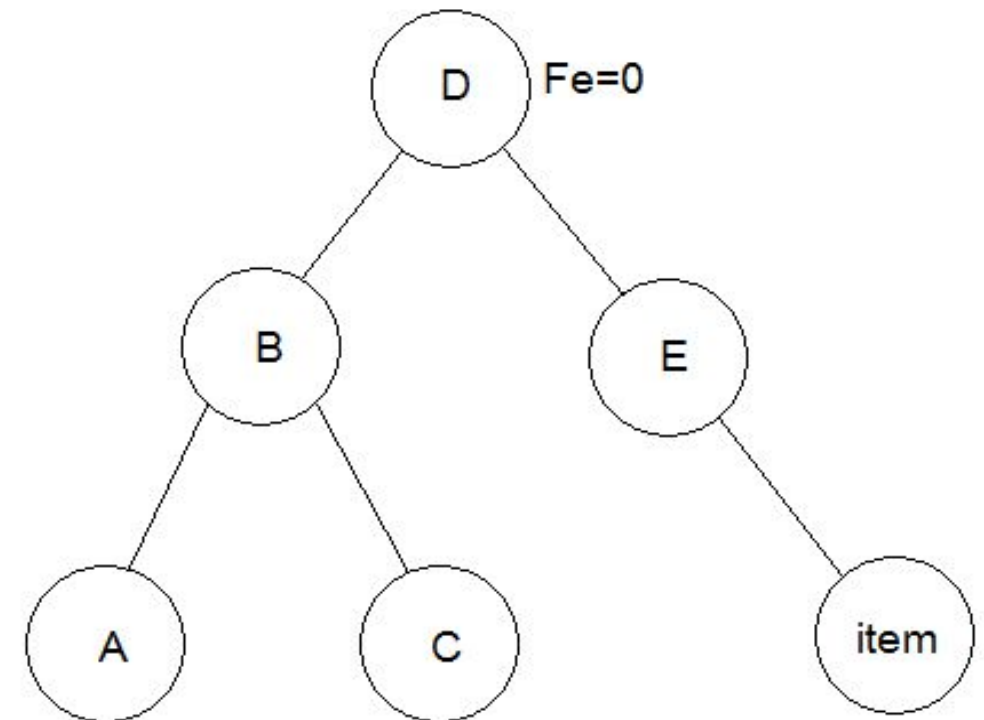
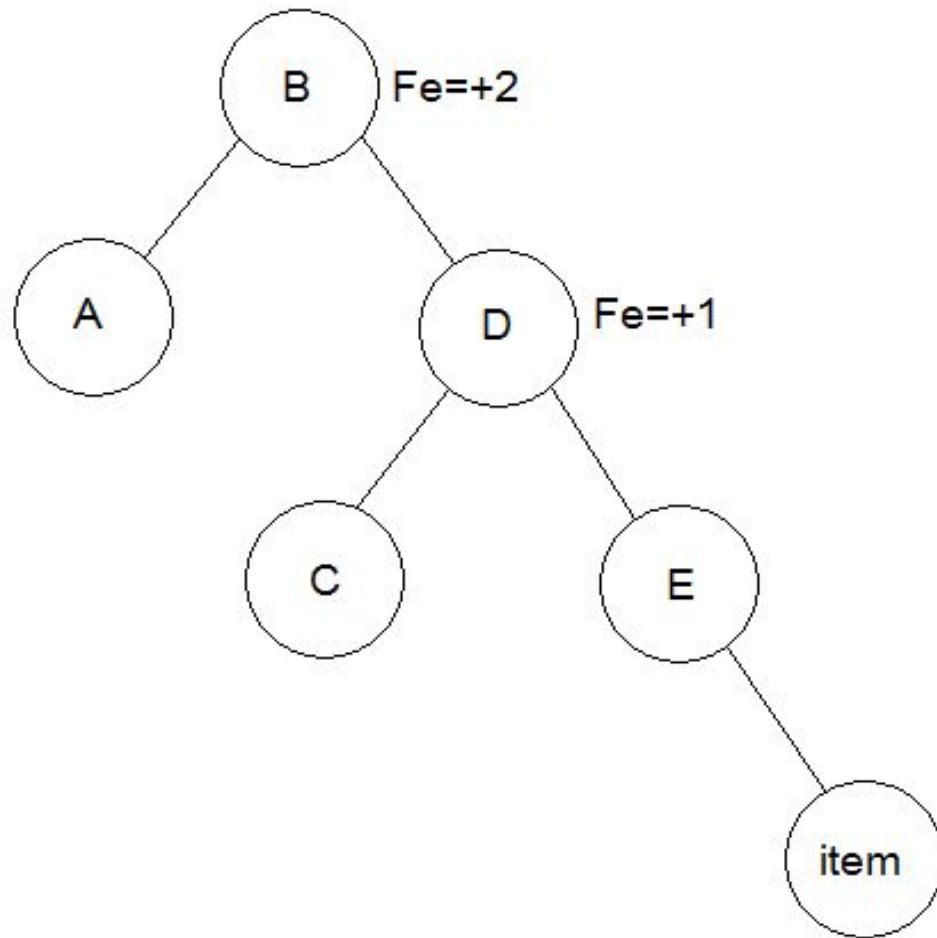
ÁRBOLES Y GRAFOS

Rotación izquierda-izquierda (II):



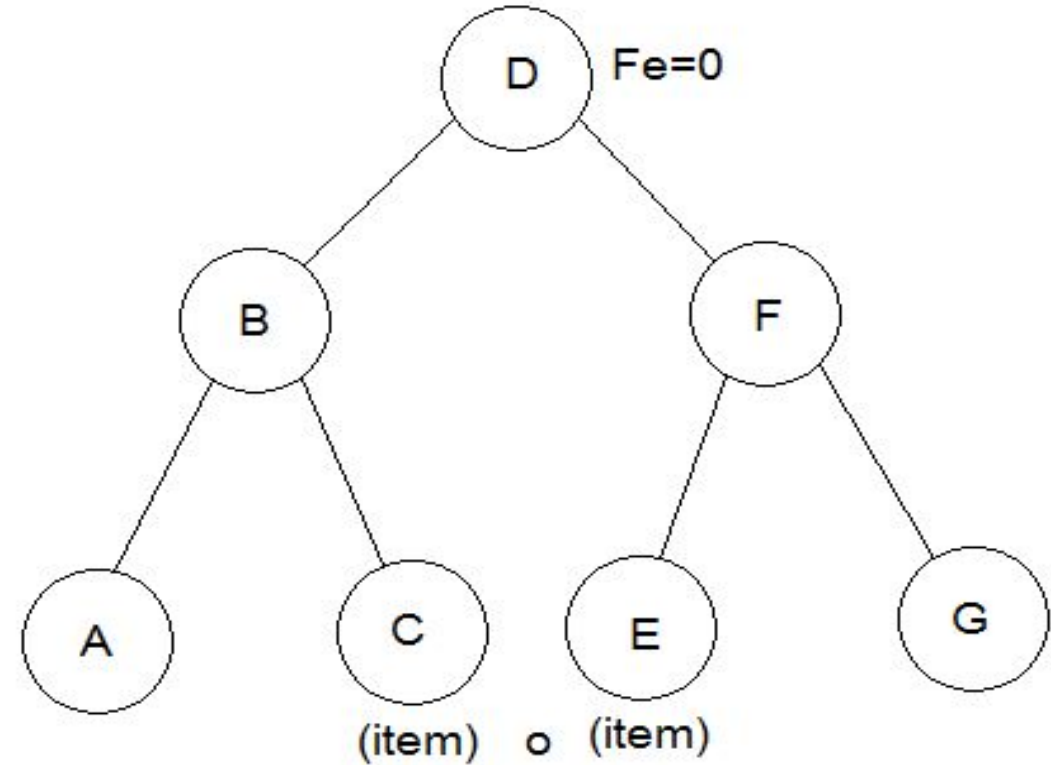
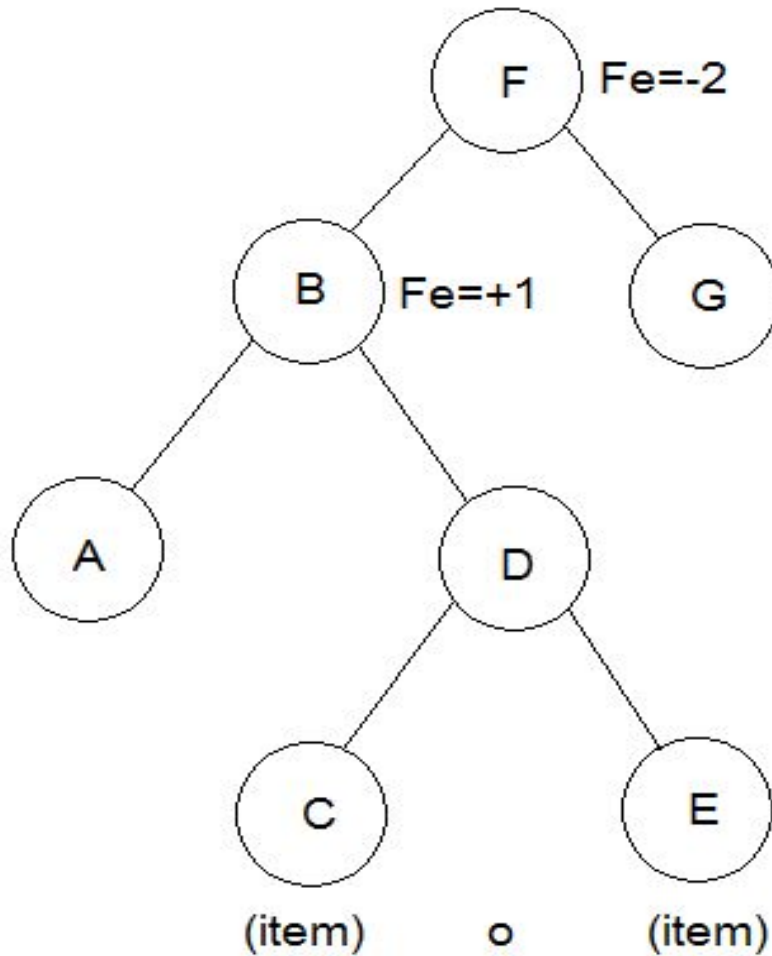
ÁRBOLES Y GRAFOS

Rotación derecha-derecha (DD):



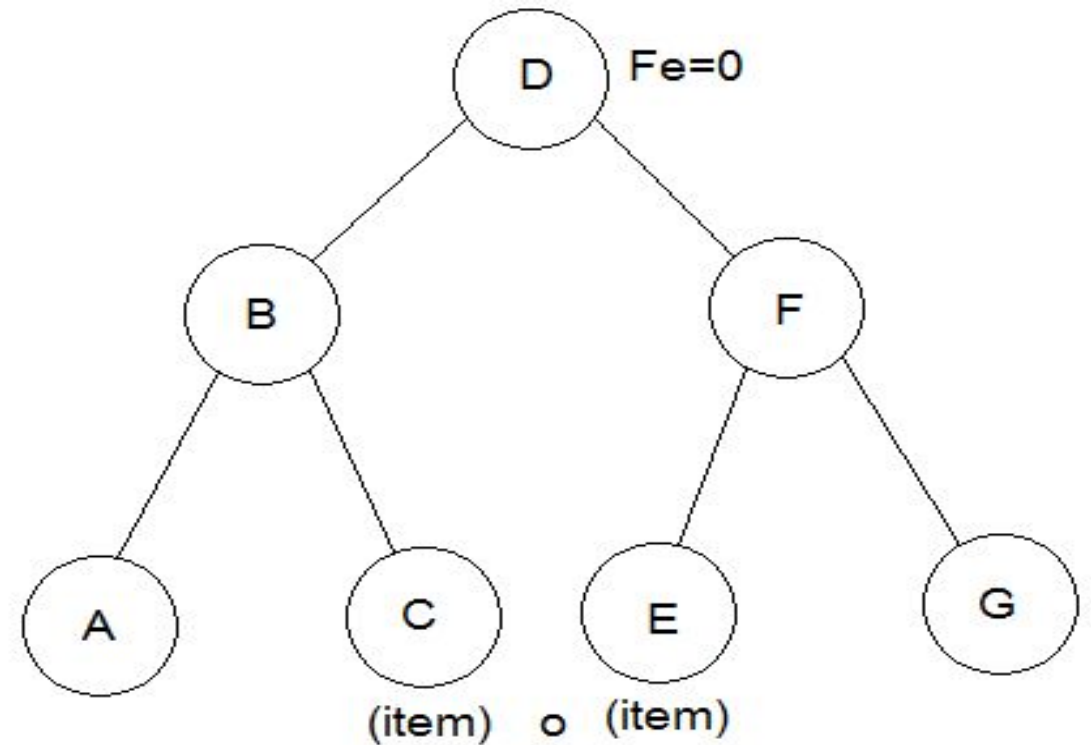
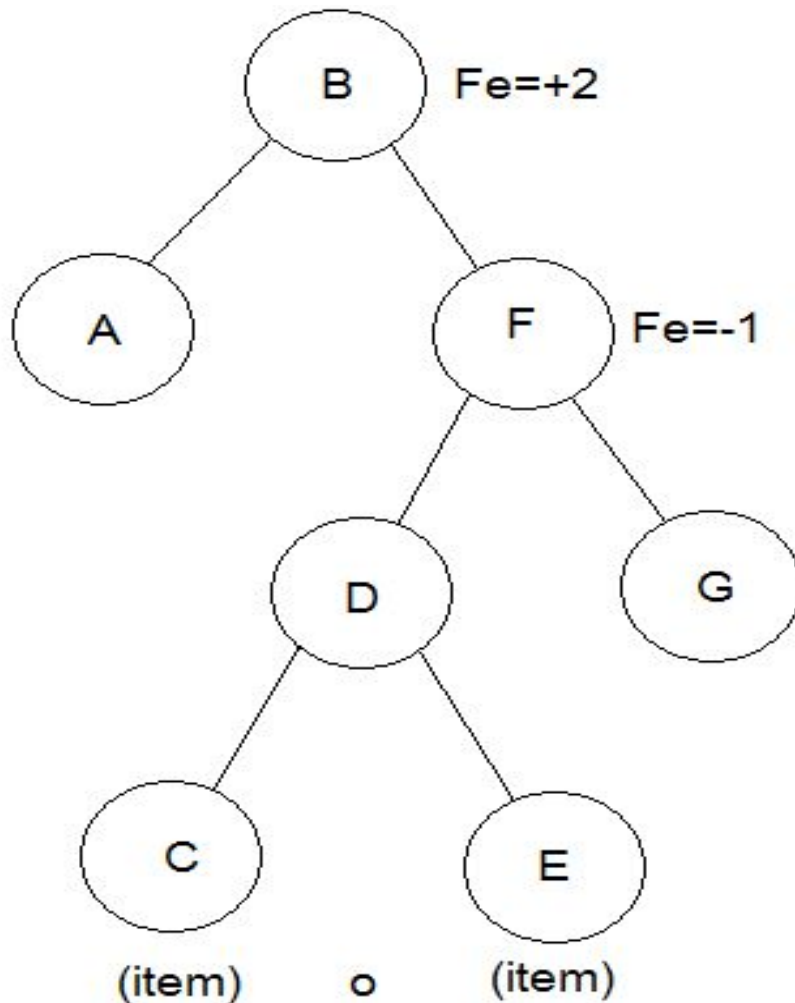
ÁRBOLES Y GRAFOS

Rotación izquierda-derecha (ID):



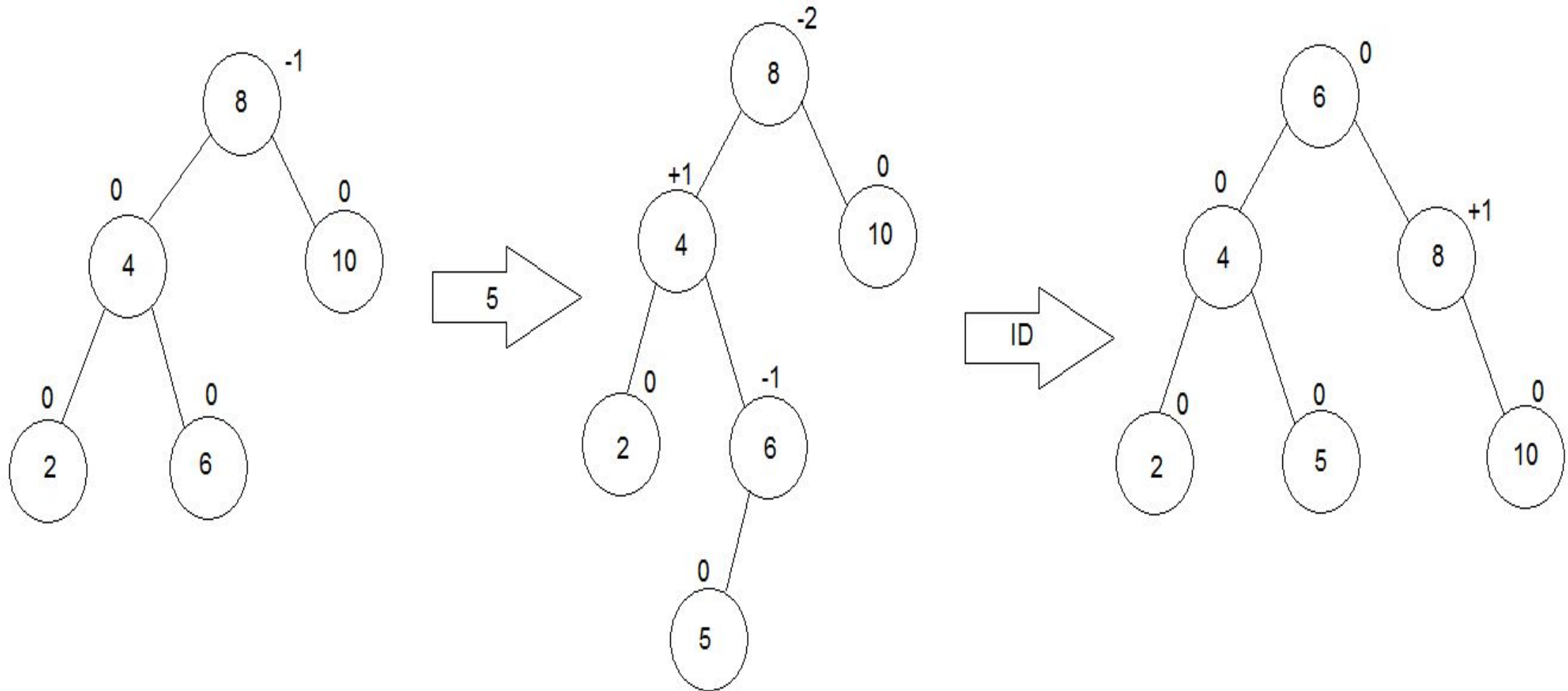
ÁRBOLES Y GRAFOS

Rotación derecha-izquierda (DI):



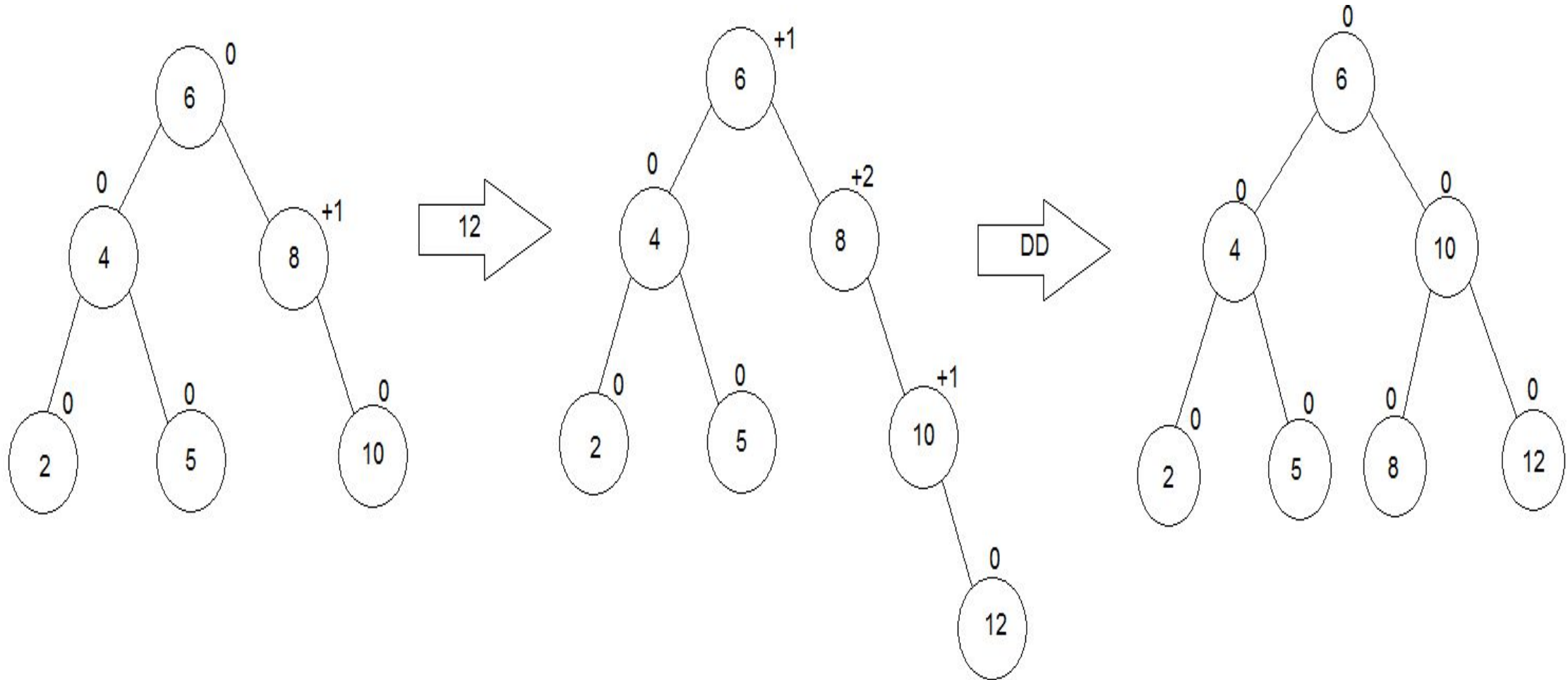
ÁRBOLES Y GRAFOS

Ejemplo:



ÁRBOLES Y GRAFOS

Ejemplo (cont.):



ÁRBOLES Y GRAFOS

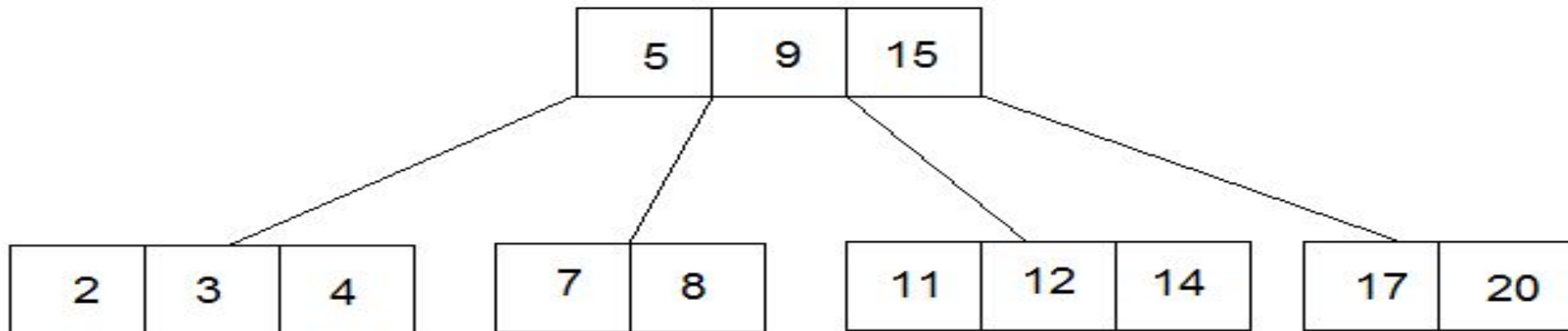
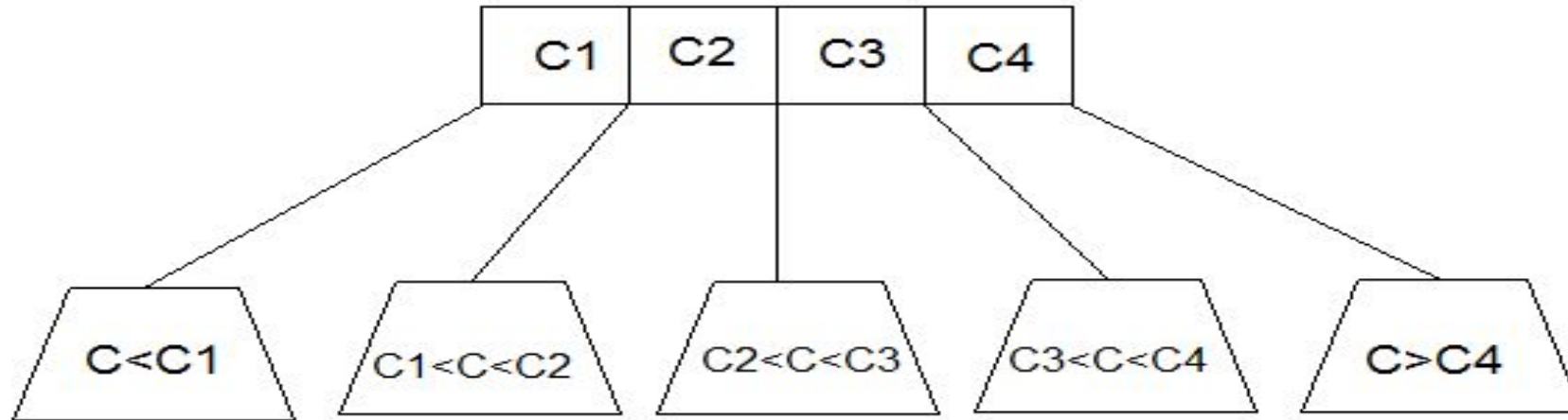
Árboles B:

Son árboles de búsqueda n-arios. Un árbol B de orden N tiene las siguientes características:

- 1) Cada nodo tiene entre $N/2$ y N hijos, salvo la raíz que puede no tener descendientes o poseer entre 2 y N hijos.
- 2) Cada nodo puede tener múltiples claves, pero siempre tiene una clave menos que la cantidad de hijos que posee.
- 3) Las claves están ordenadas y cada descendiente entre claves posee claves cuyos valores se encuentran en el rango de esas dos claves.
- 4) Todas las hojas de un árbol B están en el mismo nivel.

ÁRBOLES Y GRAFOS

Ejemplo:



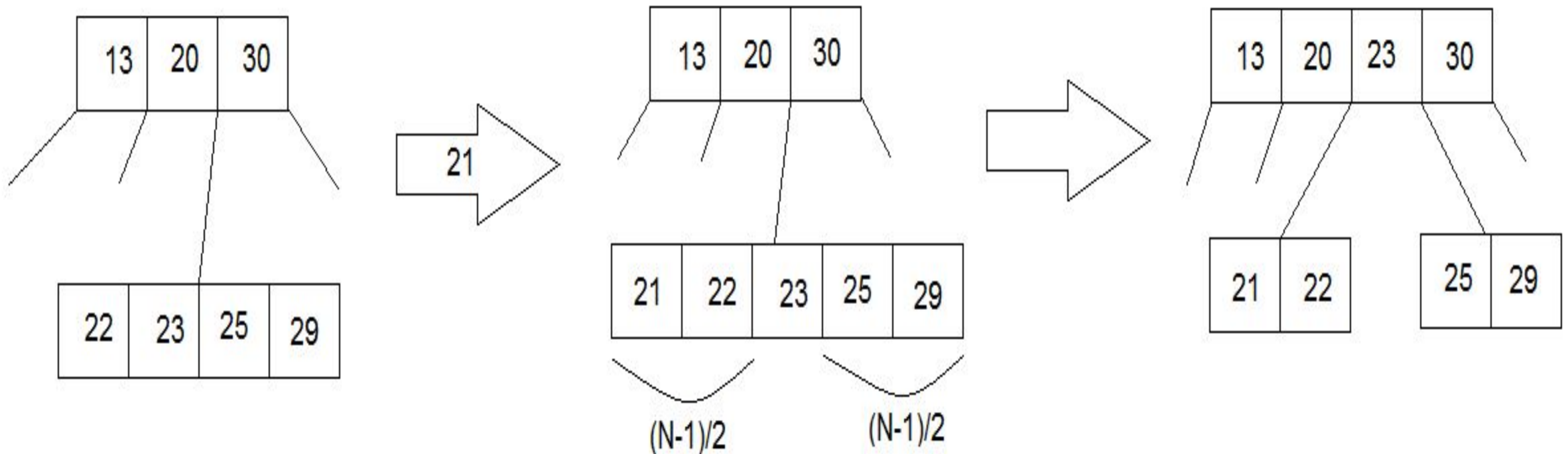
ÁRBOLES Y GRAFOS

Insertar elemento X:

- 1) Buscar el nodo hoja donde se debería colocar X, si el elemento ya está en el árbol, no se inserta.
- 2) Si la hoja tiene menos de $(N-1)$ claves, entonces hay lugar libre y se inserta en el lugar correspondiente (siguiendo el criterio de orden usado).
- 3) Si no quedan lugares libres, entonces se tendrán N claves, por lo que se particiona la hoja en dos (su padre pasará a tener un descendiente más) tomando las $((N-1)/2)$ claves primeras para formar el hijo izquierdo y las $((N-1)/2)$ claves últimas para el hijo derecho, y el valor medio subirá para ser acomodado entre las claves del padre.
- 4) Si el padre está completo, se dividirá a su vez en dos nodos, propagándose el proceso de partición hasta la raíz. Si la raíz está completa, entonces se particionará creando una nueva raíz, y la altura del árbol se incrementa en uno.

ÁRBOLES Y GRAFOS

Ejemplo con árbol B de orden N=5:



ÁRBOLES Y GRAFOS

Borrado de elemento X:

Primero debe ubicarse la clave X en el árbol B. Si está en un nodo interno, debe sustituirse la clave por el siguiente valor ordenado del árbol o el valor anterior. El valor anterior se busca como el mayor de los valores descendiendo por el hijo izquierdo de la clave X a sustituir o el menor de los valores descendiendo por el hijo derecho. Cuando la clave X queda en una hoja, se produce la eliminación pudiendo producirse uno de tres casos.

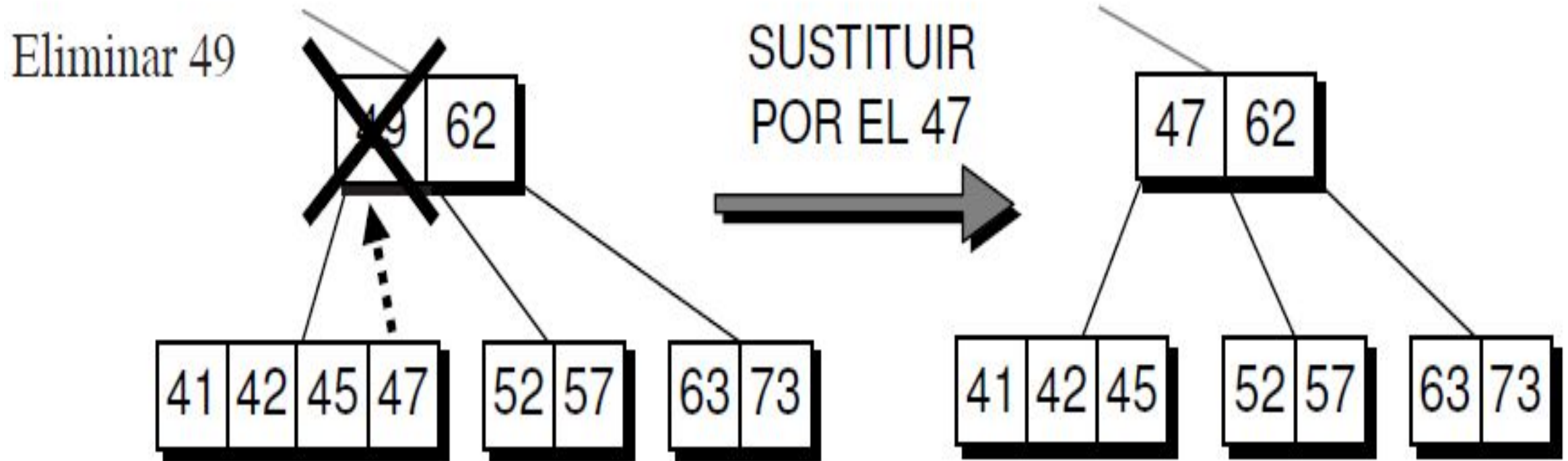
ÁRBOLES Y GRAFOS

Casos de eliminación de clave X de una hoja:

- 1) Si la hoja de supresión tiene más de $(N/2)-1$ claves, entonces se elimina X.
- 2) Si la hoja tiene $(N/2)-1$ claves y existe algún nodo hermano adyacente que tenga $(N/2)$ o más claves, entonces le hace un préstamo a la hoja de supresión: la entrada del padre común pasa a la hoja de supresión y la entrada adecuada del nodo hermano pasa a la posición del padre. Luego se elimina X.
- 3) Si la hoja tiene $(N/2)-1$ claves y los hermanos adyacentes también, entonces deberá realizarse una unión. Las claves de la hoja de supresión las claves de un hermano y la clave del padre común se forma una nueva hoja y se elimina X. Esto reduce la cantidad de claves del padre. Si quedara con menos del mínimo, se propaga el proceso hasta la raíz, reduciendo la altura del árbol en uno.

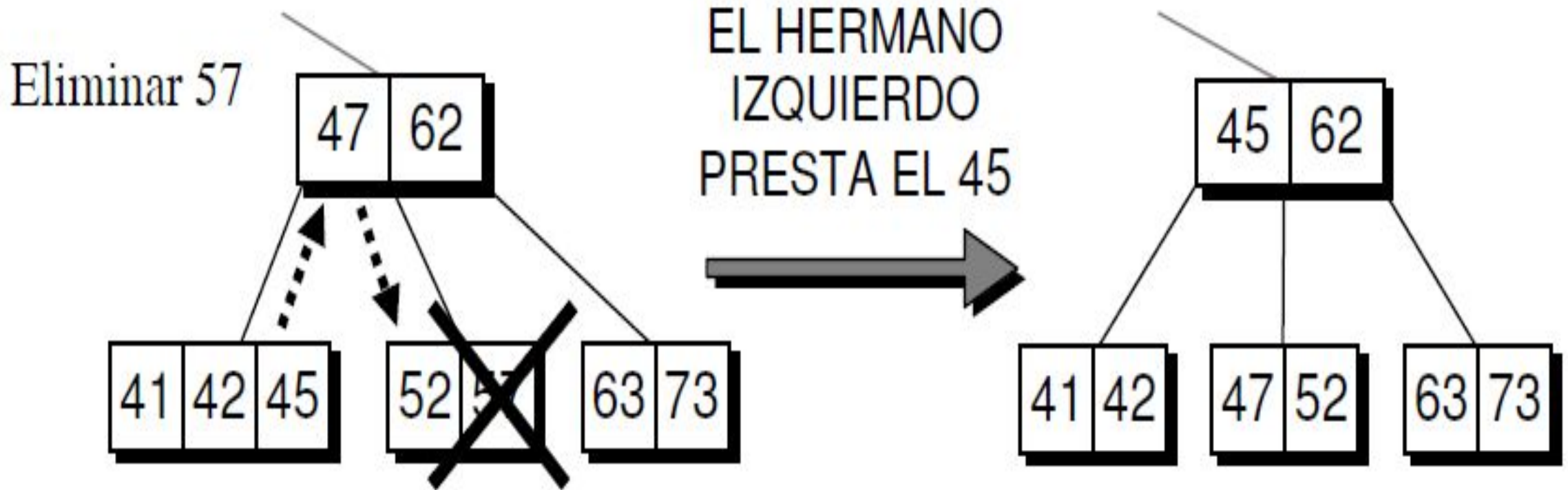
ÁRBOLES Y GRAFOS

Ejemplo 1:



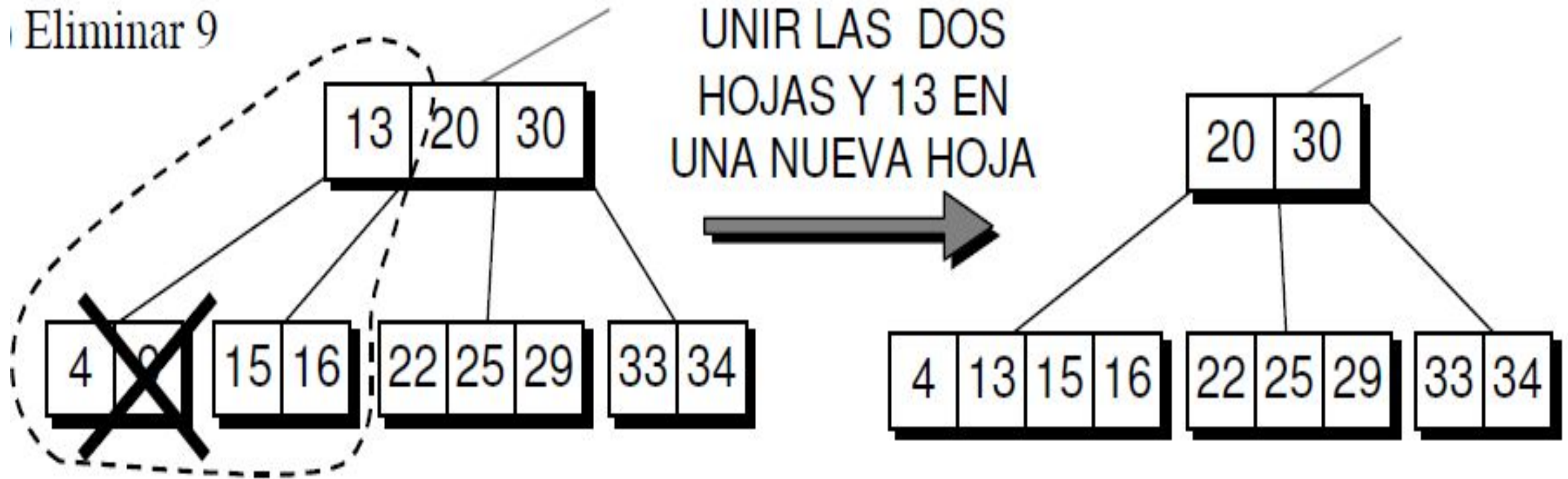
ÁRBOLES Y GRAFOS

Ejemplo 2:



ÁRBOLES Y GRAFOS

Ejemplo 3:



Árboles B+:

Son una variante de los árboles B que se emplean en la representación de diccionarios en base de datos. La estructura de los nodos hojas es distinto de los nodos internos. En los nodos internos solo aparecen las claves mientras que en las hojas hay asociaciones (clave, valor), donde valor puede ser un puntero a un registro en un archivo.

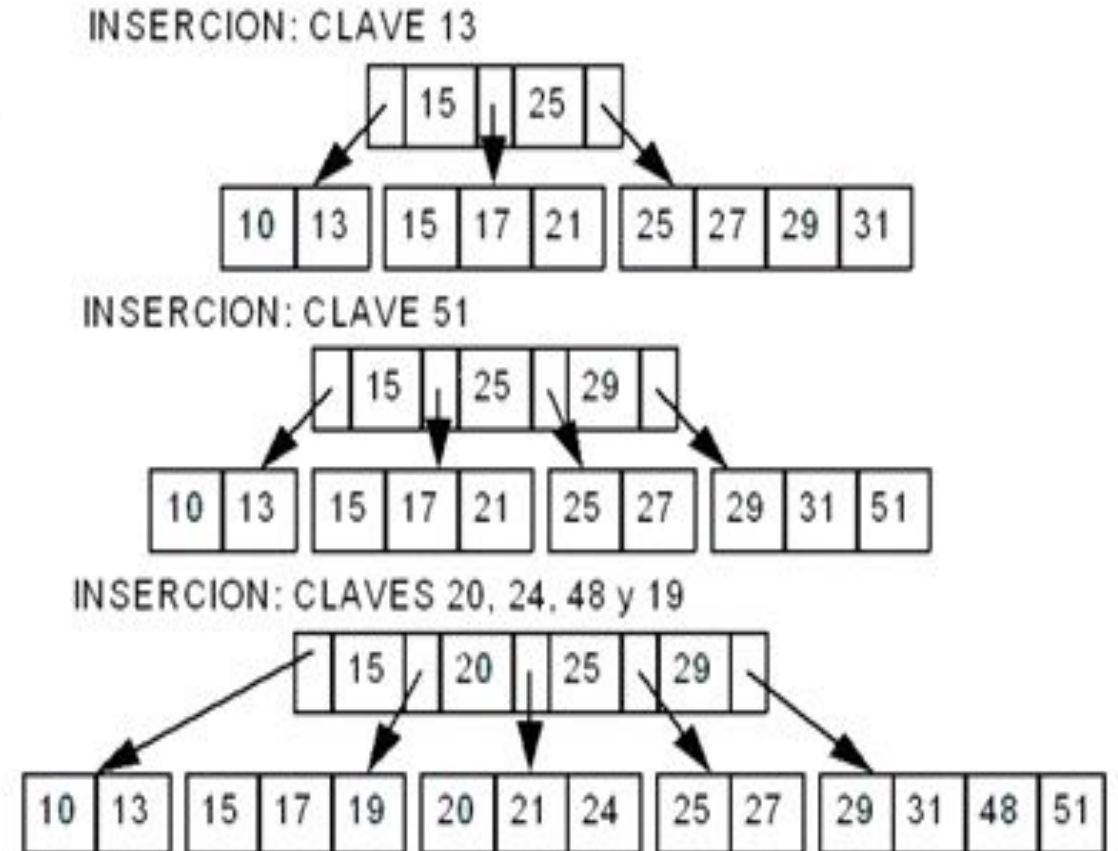
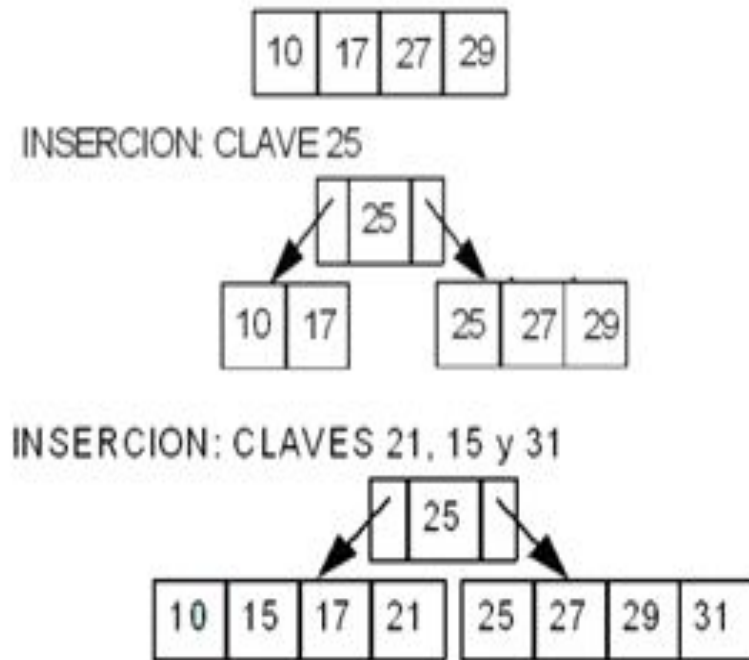
Inserción en Árboles B+:

La inserción es similar a los árboles B, se diferencia en que cuando debe realizarse la partición de un nodo hoja, se sube un duplicado de la clave media cuando la clave media es de un nodo hoja, manteniendo la clave media con su valor asociado en una de las hojas particionadas.

El resto de los casos es como un árbol B.

ÁRBOLES Y GRAFOS

Ejemplo:



ÁRBOLES Y GRAFOS

Eliminación en Árboles B+:

La eliminación es similar a los árboles B, solo que es más simple, ya que siempre se elimina una clave de un nodo hoja.

Acceso secuencial en Árboles B+:

Un aspecto a tener en cuenta en árboles B+ es que debe proporcionarse una forma ágil de acceso secuencial. Esto se consigue haciendo que cada nodo hoja tenga un puntero al hermano derecho adyacente.

Árboles B*:

Son una variación de los árboles B. Si en un árbol B los nodos deben estar ocupados como mínimo hasta la mitad del orden que poseen, en los árboles B* tienen que estar ocupados más de dos tercios del máximo.

Este hecho mejora considerablemente el aprovechamiento de espacio en memoria a comparación de los árboles B.

ÁRBOLES Y GRAFOS

GRAFOS:

Un grafo es una estructura de datos formado por un conjunto de nodos o vértices y un conjunto de arcos o aristas que se especifican por el par de nodos que unen. Los arcos pueden ser dirigidos o no.

Al número de nodos de un grafo se lo llama ORDEN del grafo.

Dos nodos son ADYACENTES si hay un arco que los une.

Un CAMINO es una secuencia de uno o más arcos que conecta dos nodos.

Un grafo es CONEXO si existe siempre un camino entre cualquier par de nodos.

ÁRBOLES Y GRAFOS

GRAFOS (cont.):

Un grafo es COMPLETO cuando cada nodo está conectado con todos y cada uno de los nodos restantes.

El camino de un nodo a sí mismo se denomina CICLO.

El GRADO de un vértice es la cantidad de aristas que llegan a él.

Un grafo es REGULAR si todos sus vértices tienen el mismo grado.

La LONGITUD de un camino se mide por la cantidad de aristas que lo componen. En caso de tratarse de aristas con peso (etiquetadas), deben sumarse los pesos de todos los arcos que conforman el camino para obtener la DISTANCIA.

ÁRBOLES Y GRAFOS

GRAFOS (cont. 2):

Un ciclo Hamiltoniano es un ciclo que contiene todos los vértices del grafo.

Un camino Hamiltoniano es un camino (no cerrado) que une todos los vértices del grafo.

Un grafo que tiene un ciclo Hamiltoniano es un grafo Hamiltoniano.

Dados dos nodos cualesquiera de un grafo, pueden existir múltiples caminos entre ambos, pero solamente uno será el más óptimo, es decir, el que posea la menor distancia. Este camino se conoce como CAMINO MÍNIMO.

ÁRBOLES Y GRAFOS

GRAFOS (cont. 3):

Un grafo pesado o un grafo con peso, es aquel cuyas aristas están etiquetadas con un peso o costo.

El costo de una arista puede representar el costo de movilizarse entre el par de nodos que unen.

Por ejemplo, los vértices podrían tratarse de ciudades y las aristas de rutas que las unen. El peso de cada arista será la distancia en Km que hay entre las ciudades que une el arco que representa la ruta.

Algoritmo de Dijkstra:

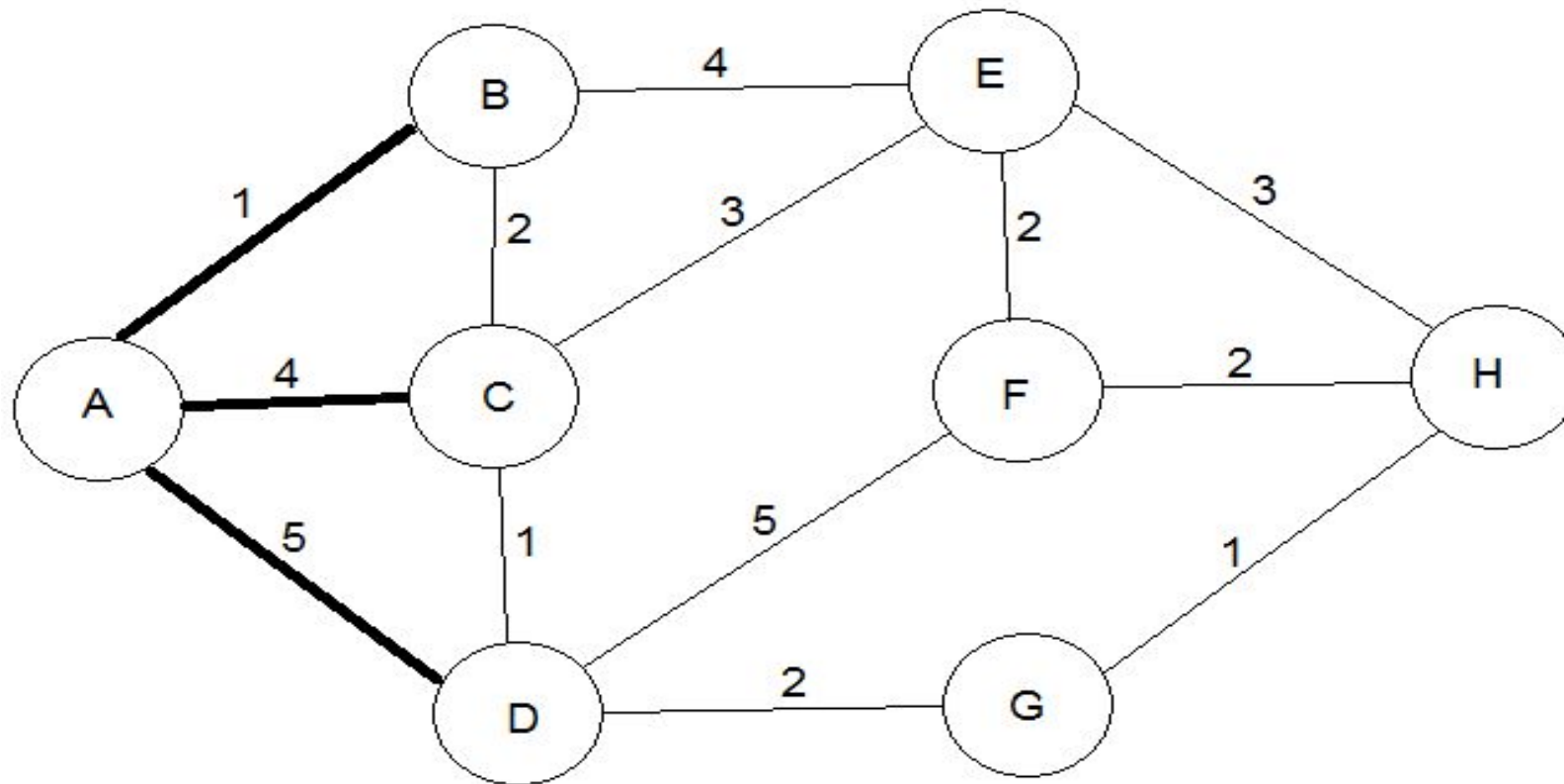
Este algoritmo encuentra TODOS los caminos mínimos de un nodo cualquiera de un grafo al RESTO de los nodos del grafo.

Algoritmo:

- 1) Pararse en el nodo inicial
- 2) Visitar los nodos adyacentes
- 3) Marcar el menor
- 4) Desde el menor visitar los adyacentes
- 5) Mejorar los caminos que se puedan desde el nuevo nodo
- 6) Marcar el menor
- 7) Repetir hasta que no queden nodos por marcar

ÁRBOLES Y GRAFOS

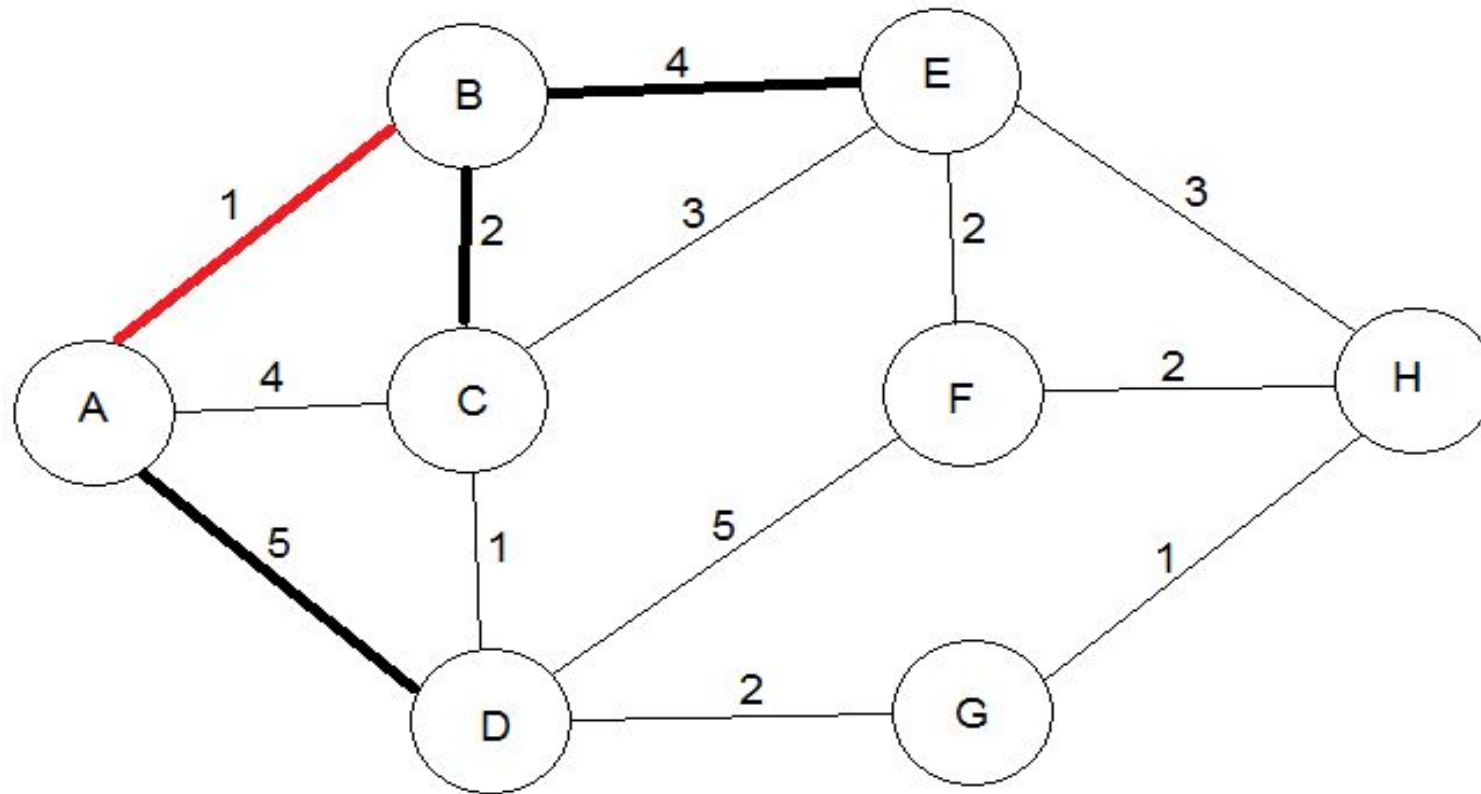
Ejemplo 1:



	1 (A)
A	(0, A)
B	(1, A)
C	(4, A)
D	(5, A)
E	--
F	--
G	--
H	--

ÁRBOLES Y GRAFOS

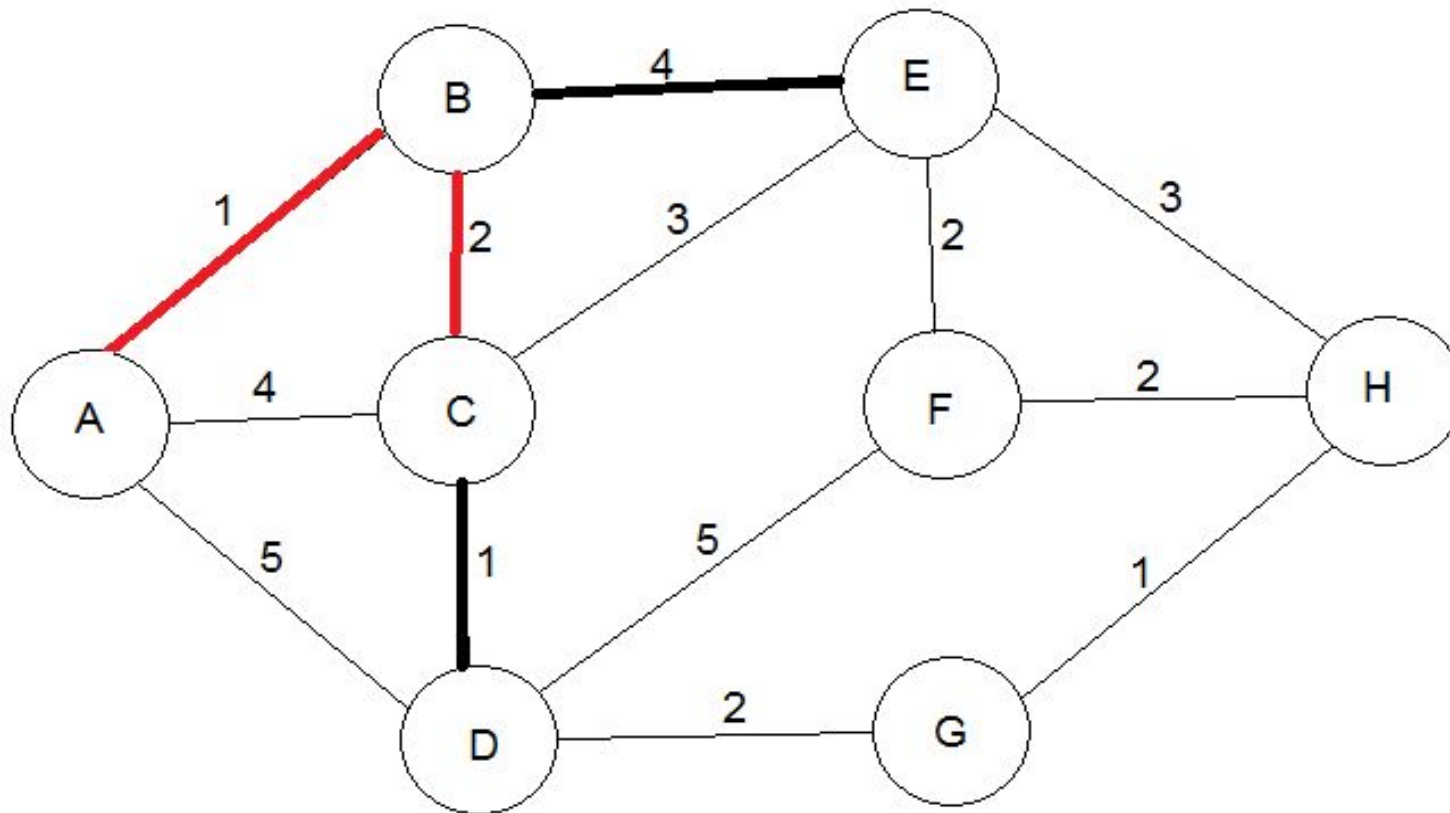
Ejemplo 1 (cont.):



	1 (A)	2 (B)
A	(0, A)	X
B	(1, A)	(1, A)
C	(4, A)	(3, B)
D	(5, A)	(5, A)
E	--	(5, B)
F	--	--
G	--	--
H	--	--

ÁRBOLES Y GRAFOS

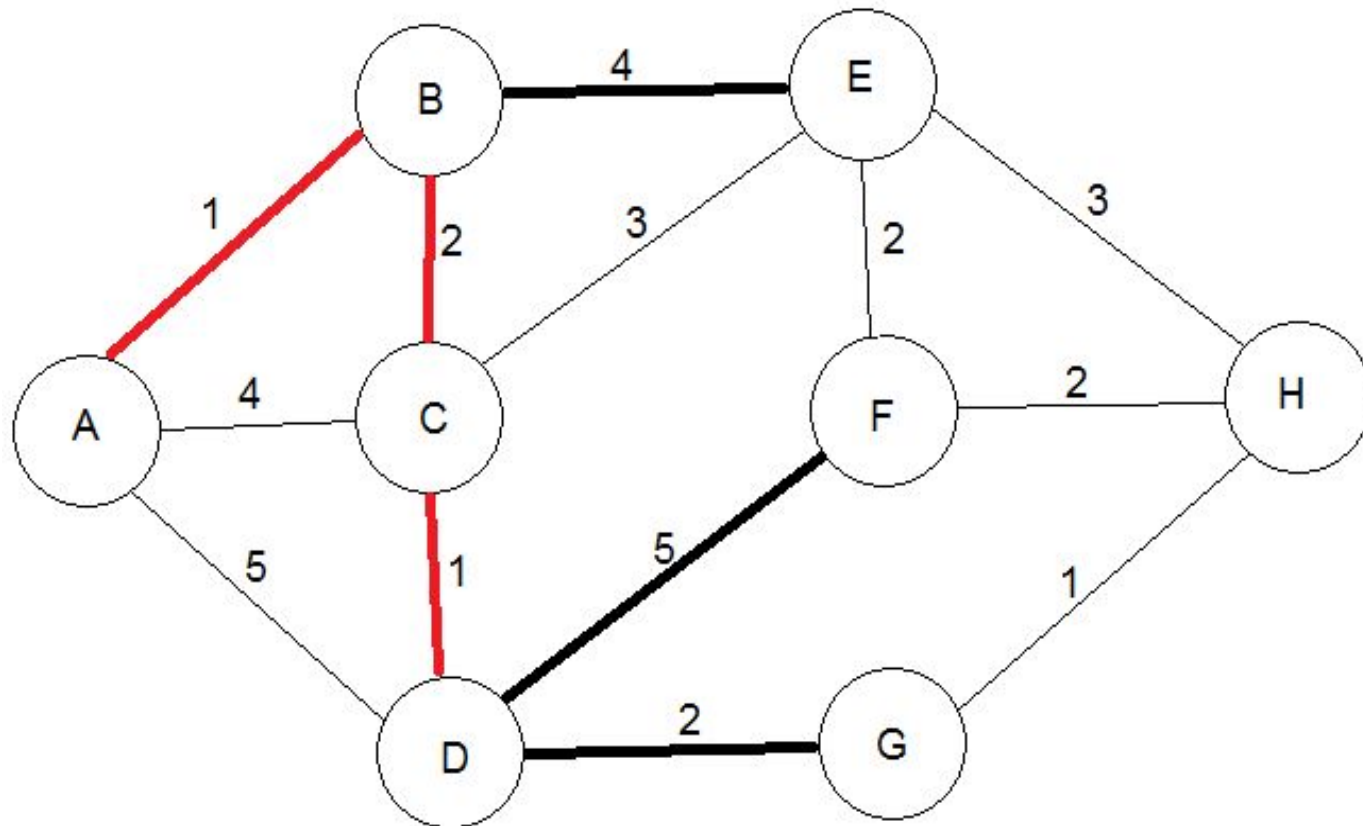
Ejemplo 1 (cont. 2):



	1 (A)	2 (B)	3 (C)
A	(0, A)	X	X
B	(1, A)	(1, A)	X
C	(4, A)	(3, B)	(3, B)
D	(5, A)	(5, A)	(4, C)
E	--	(5, B)	(5, B)
F	--	--	--
G	--	--	--
H	--	--	--

ÁRBOLES Y GRAFOS

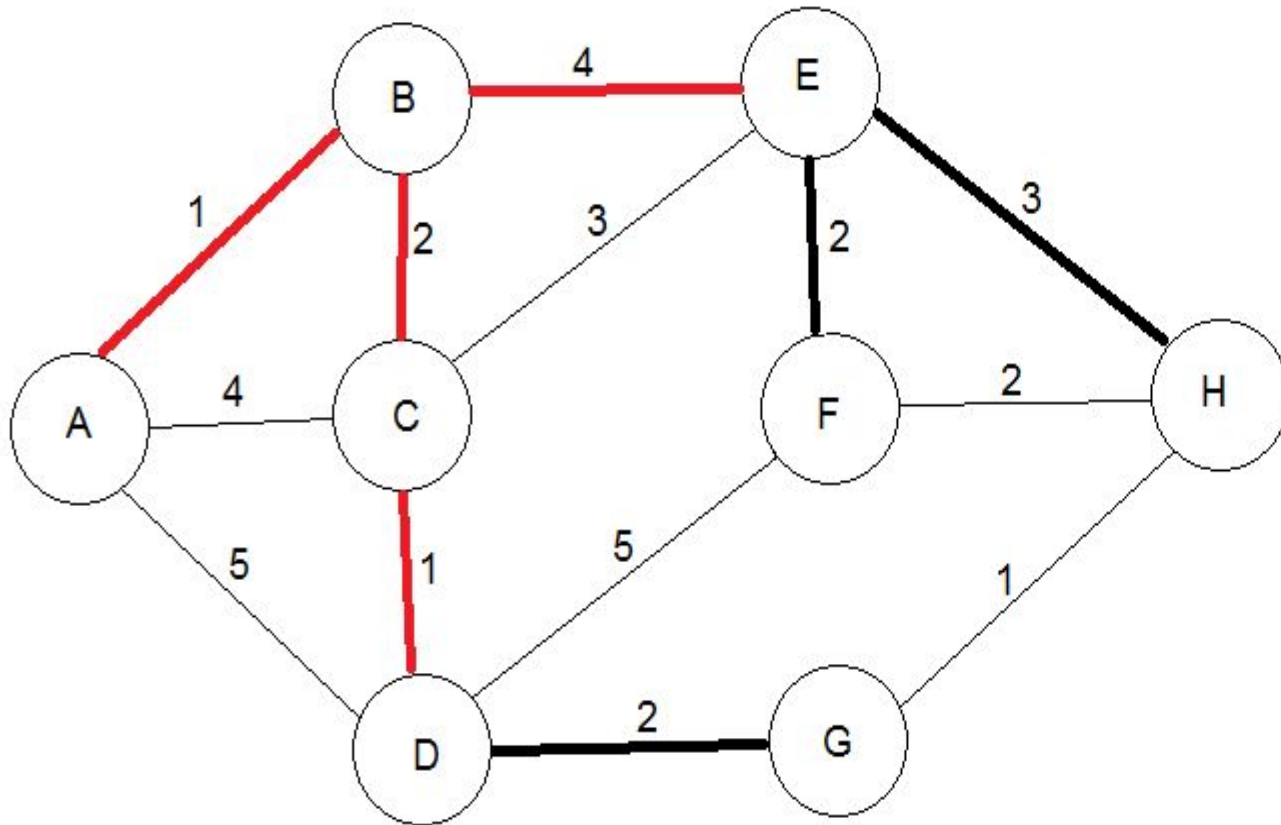
Ejemplo 1 (cont. 3):



	1 (A)	2 (B)	3 (C)	4 (D)
A	(0, A)	X	X	X
B	(1, A)	(1, A)	X	X
C	(4, A)	(3, B)	(3, B)	X
D	(5, A)	(5, A)	(4, C)	(4, C)
E	--	(5, B)	(5, B)	(5, B)
F	--	--	--	(9, D)
G	--	--	--	(6, D)
H	--	--	--	--

ÁRBOLES Y GRAFOS

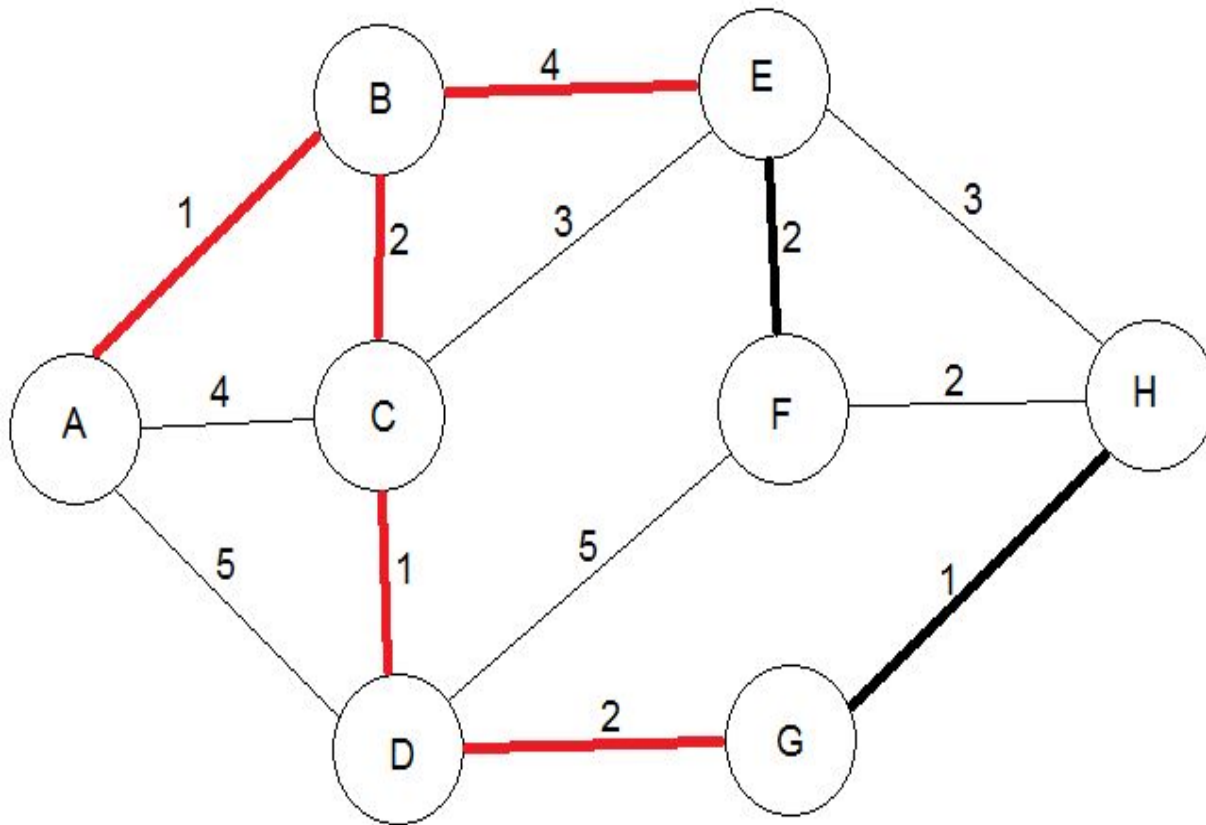
Ejemplo 1 (cont. 4):



	1 (A)	2 (B)	3 (C)	4 (D)	5 (E)
A	(0, A)	X	X	X	X
B	(1, A)	(1, A)	X	X	X
C	(4, A)	(3, B)	(3, B)	X	X
D	(5, A)	(5, A)	(4, C)	(4, C)	X
E	--	(5, B)	(5, B)	(5, B)	(5, B)
F	--	--	--	(9, D)	(7, E)
G	--	--	--	(6, D)	(6, D)
H	--	--	--	--	(8, E)

ÁRBOLES Y GRAFOS

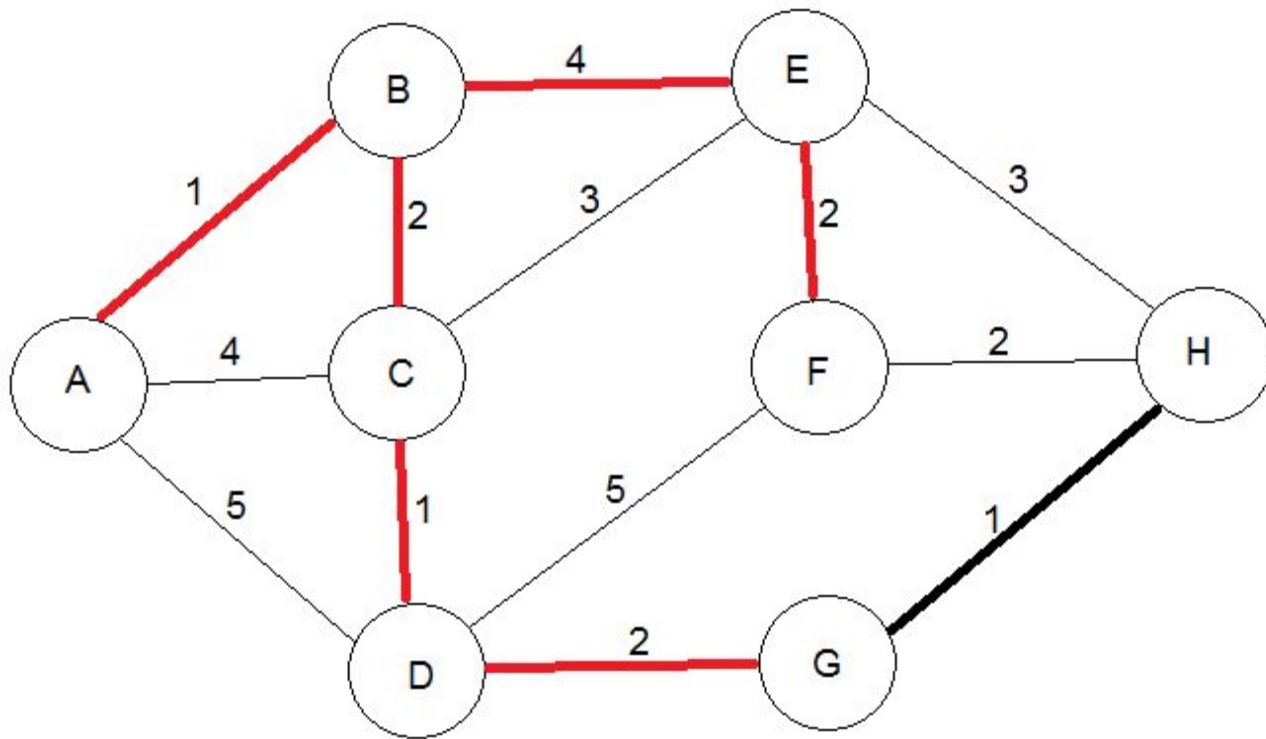
Ejemplo 1 (cont. 5):



	1 (A)	2 (B)	3 (C)	4 (D)	5 (E)	6 (G)
A	(0, A)	X	X	X	X	X
B	(1, A)	(1, A)	X	X	X	X
C	(4, A)	(3, B)	(3, B)	X	X	X
D	(5, A)	(5, A)	(4, C)	(4, C)	X	X
E	--	(5, B)	(5, B)	(5, B)	(5, B)	X
F	--	--	--	(7, E)	(7, E)	(7, E)
G	--	--	--	(6, D)	(6, D)	(6, D)
H	--	--	--	--	(8, E)	(7, G)

ÁRBOLES Y GRAFOS

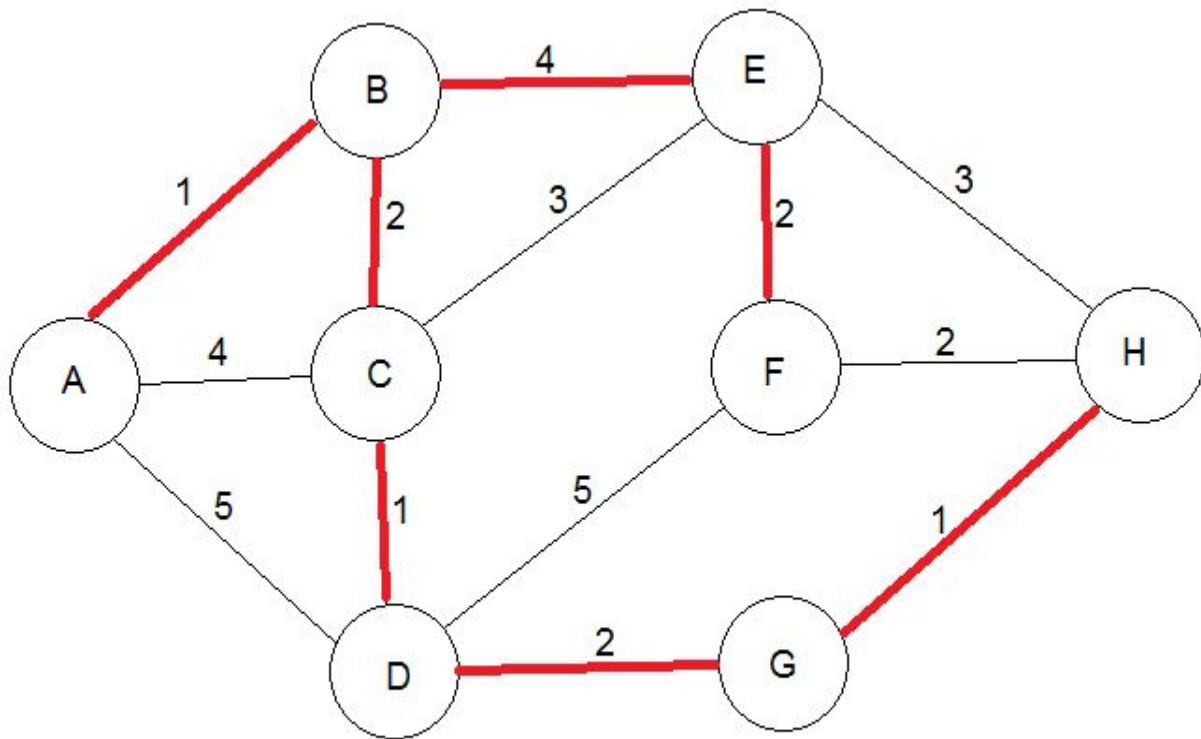
Ejemplo 1 (cont. 6):



	1 (A)	2 (B)	3 (C)	4 (D)	5 (E)	6 (G)	7 (F)
A	(0, A)	X	X	X	X	X	X
B	(1, A)	(1, A)	X	X	X	X	X
C	(4, A)	(3, B)	(3, B)	X	X	X	X
D	(5, A)	(5, A)	(4, C)	(4, C)	X	X	X
E	--	(5, B)	(5, B)	(5, B)	(5, B)	X	X
F	--	--	--	(7, E)	(7, E)	(7, E)	(7, E)
G	--	--	--	(6, D)	(6, D)	(6, D)	X
H	--	--	--	--	(8, E)	(7, G)	(7, G)

ÁRBOLES Y GRAFOS

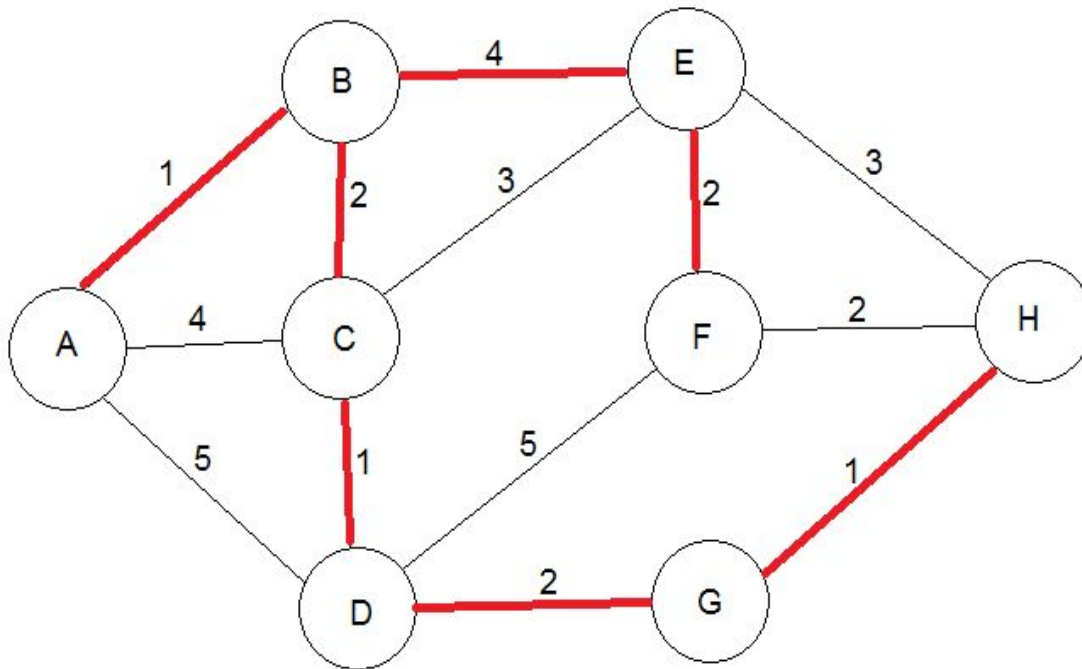
Ejemplo 1 (cont. 7):



	1 (A)	2 (B)	3 (C)	4 (D)	5 (E)	6 (G)	7 (F)	8 (H)
A	(0, A)	X	X	X	X	X	X	X
B	(1, A)	(1, A)	X	X	X	X	X	X
C	(4, A)	(3, B)	(3, B)	X	X	X	X	X
D	(5, A)	(5, A)	(4, C)	(4, C)	X	X	X	X
E	--	(5, B)	(5, B)	(5, B)	(5, B)	X	X	X
F	--	--	--	(7, E)	(7, E)	(7, E)	(7, E)	X
G	--	--	--	(6, D)	(6, D)	(6, D)	X	X
H	--	--	--	--	(8, E)	(7, G)	(7, G)	(7, G)

ÁRBOLES Y GRAFOS

Ejemplo 1 (cont. 8):



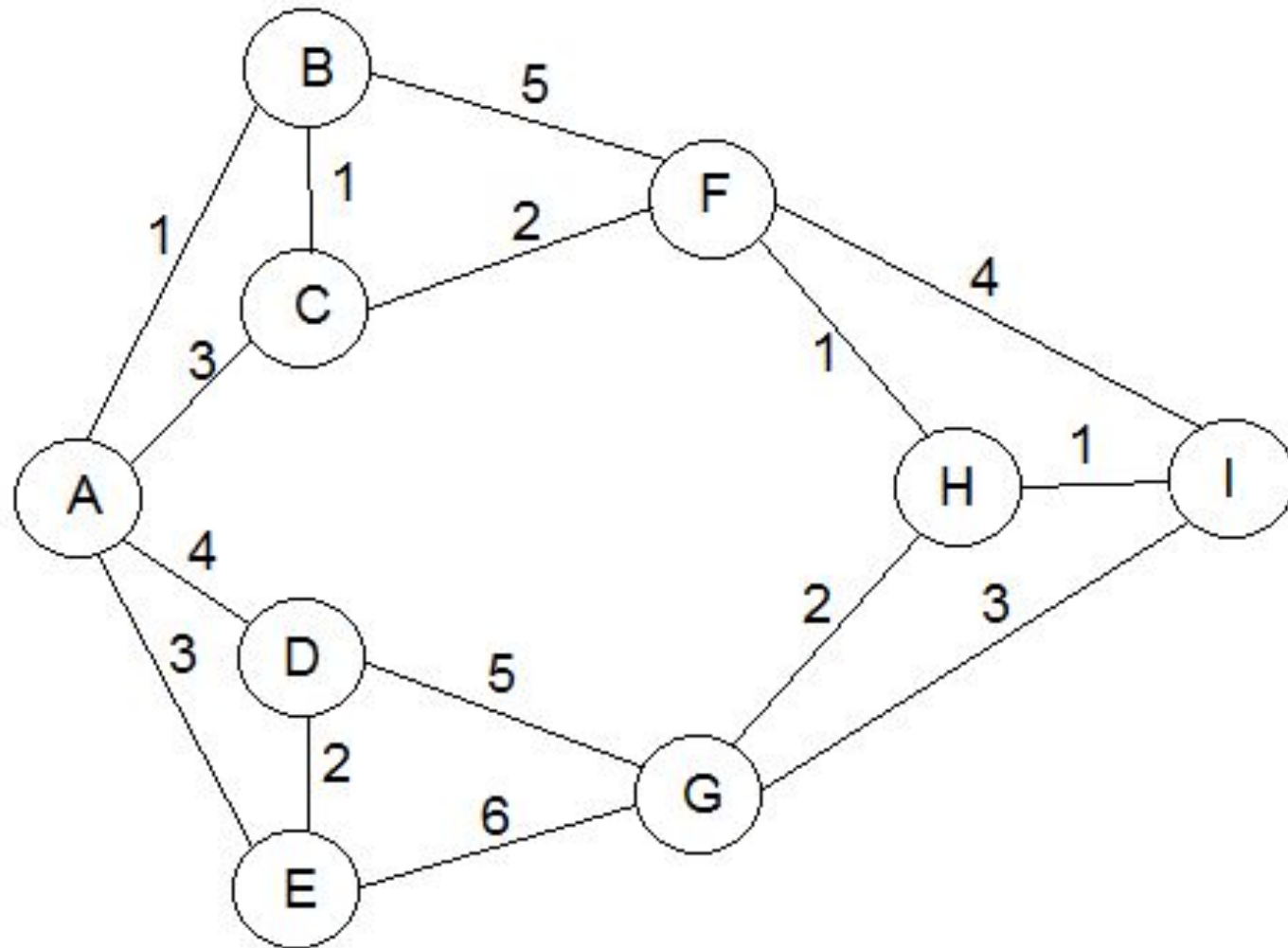
	1 (A)	2 (B)	3 (C)	4 (D)	5 (E)	6 (G)	7 (F)	8 (H)
A	(0, A)	X	X	X	X	X	X	X
B	(1, A)	(1, A)	X	X	X	X	X	X
C	(4, A)	(3, B)	(3, B)	X	X	X	X	X
D	(5, A)	(5, A)	(4, C)	(4, C)	X	X	X	X
E	--	(5, B)	(5, B)	(5, B)	(5, B)	X	X	X
F	--	--	--	(7, E)	(7, E)	(7, E)	(7, E)	X
G	--	--	--	(6, D)	(6, D)	(6, D)	X	X
H	--	--	--	--	(8, E)	(7, G)	(7, G)	(7, G)

Camino de A->G: G->(6, D)->(4,C)->(3, B)->(1, A)

A¹->B³->C¹->D⁶->G⁴ (el camino mínim de A hasta G tiene un costo de 6: (6, D))

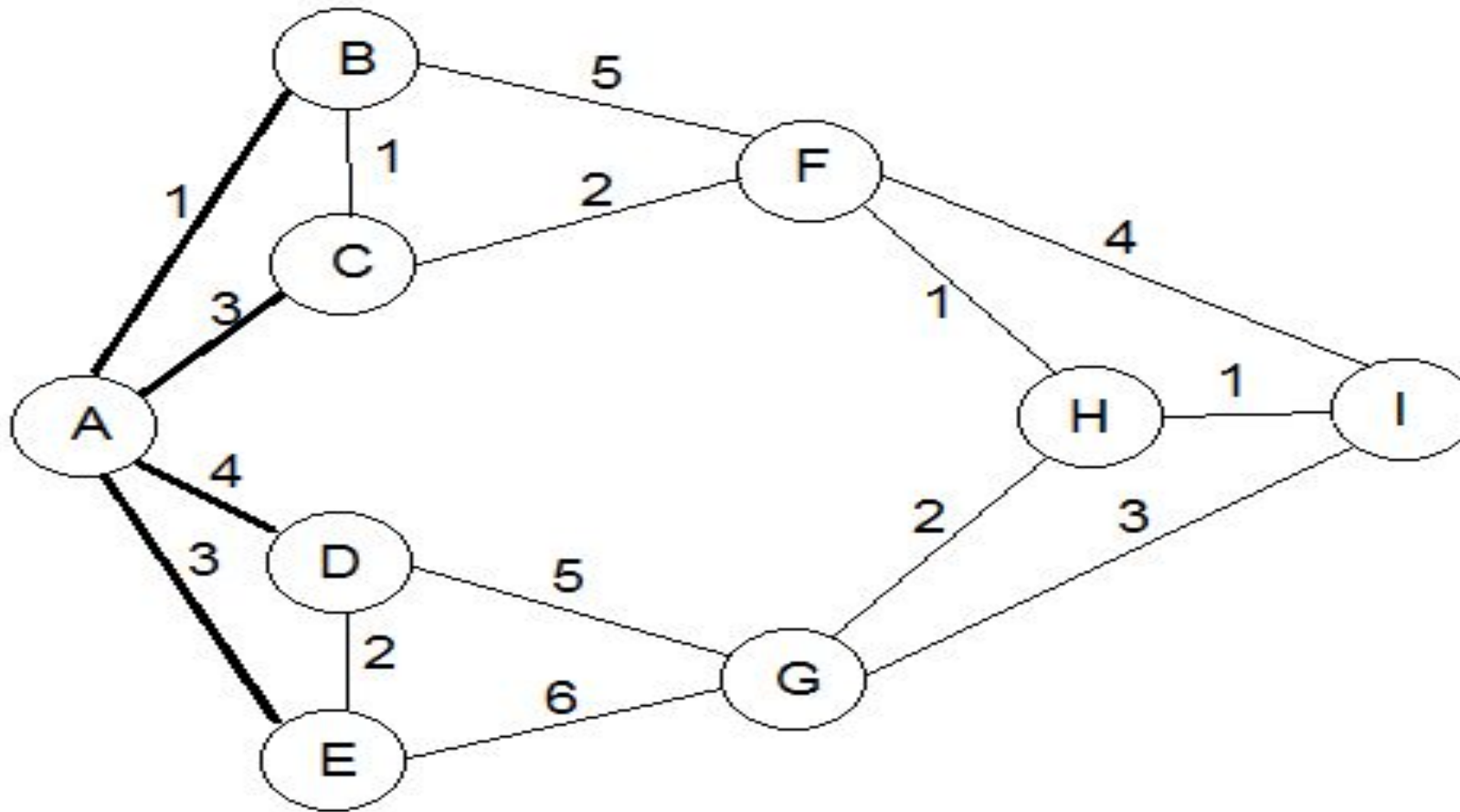
ÁRBOLES Y GRAFOS

Ejemplo 2:



ÁRBOLES Y GRAFOS

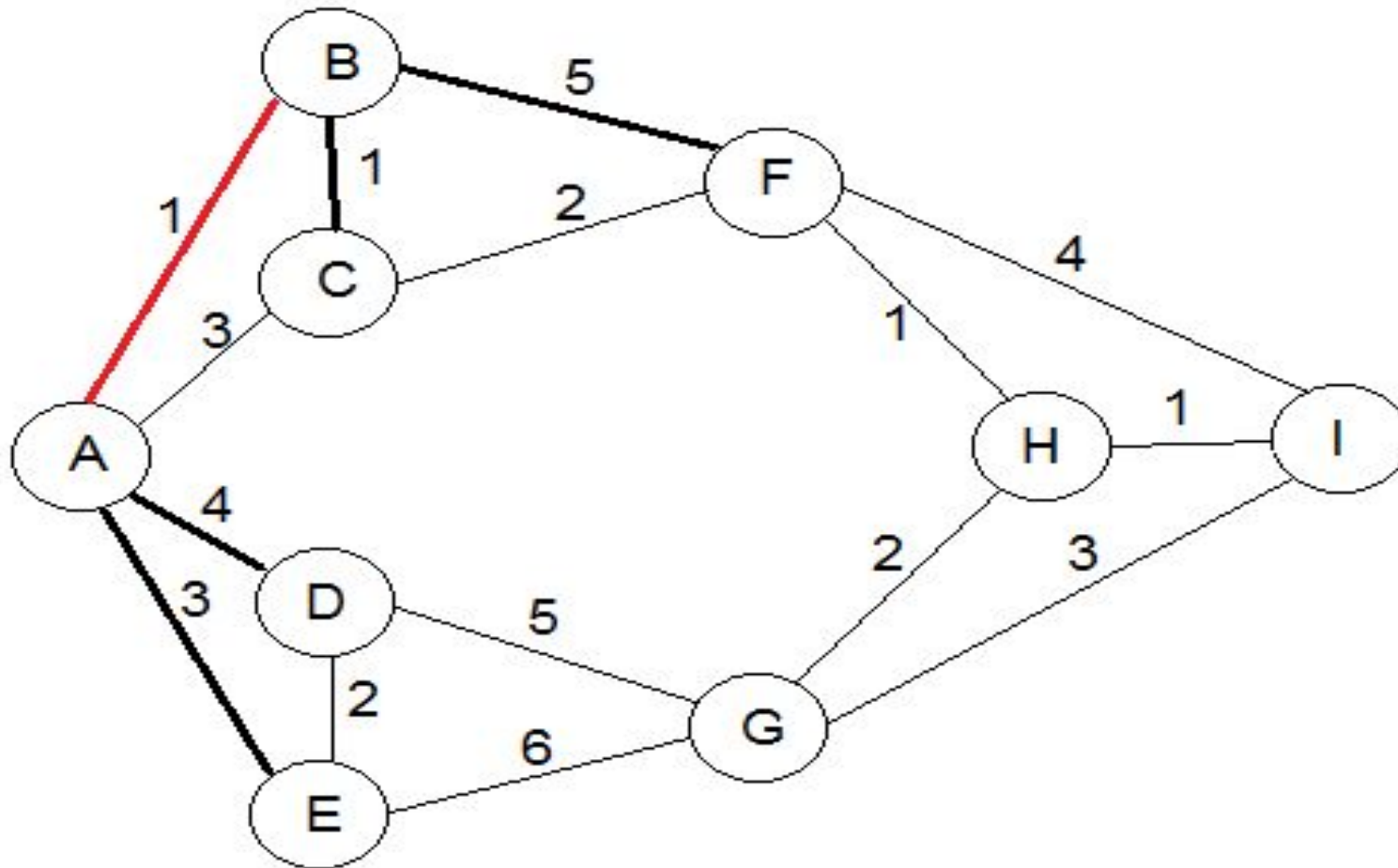
Ejemplo 2 (cont.):



	1 (A)
A	(0, A)
B	(1, A)
C	(3, A)
D	(4, A)
E	(3, A)
F	---
G	---
H	---
I	---

ÁRBOLES Y GRAFOS

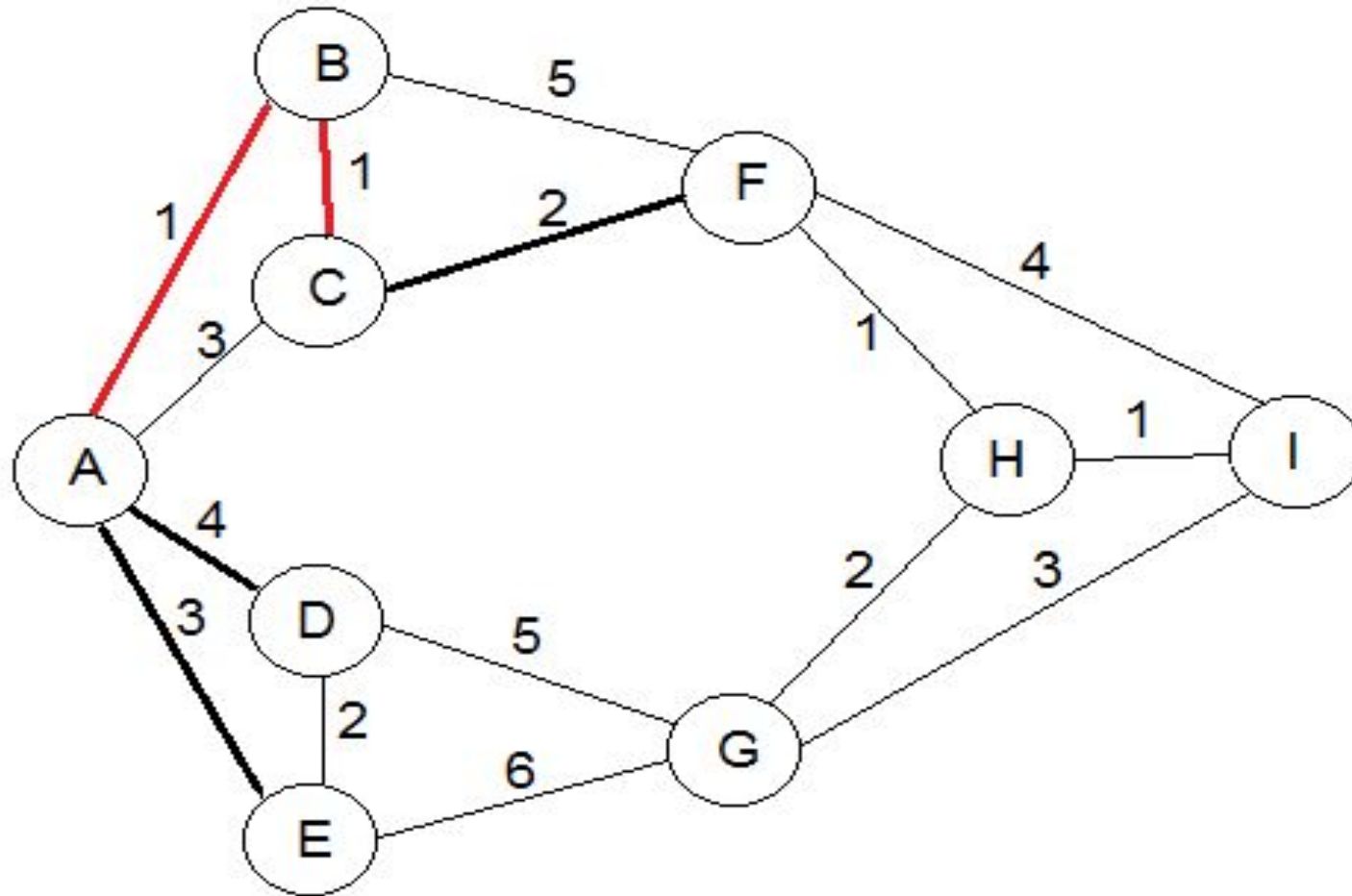
Ejemplo 2 (cont. 2):



	1 (A)	2 (B)
A	(0, A)	X
B	(1, A)	(1, A)
C	(3, A)	(2, B)
D	(4, A)	(4, A)
E	(3, A)	(3, A)
F	---	(6, B)
G	---	---
H	---	---
I	---	---

ÁRBOLES Y GRAFOS

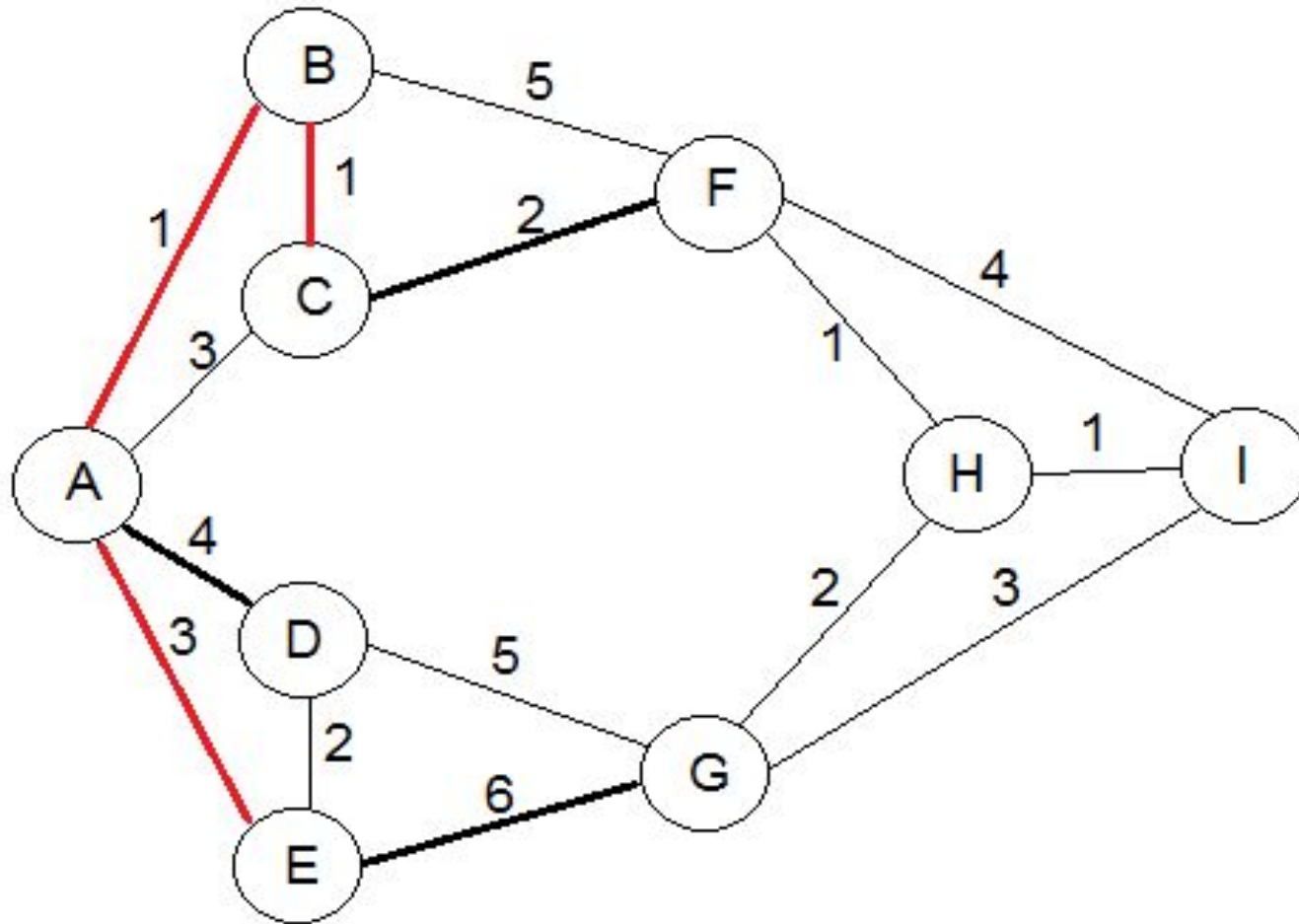
Ejemplo 2 (cont. 3):



	1 (A)	2 (B)	3 (C)
A	(0, A)	X	X
B	(1, A)	(1, A)	X
C	(3, A)	(2, B)	(2, B)
D	(4, A)	(4, A)	(4, A)
E	(3, A)	(3, A)	(3, A)
F	---	(6, B)	(4, C)
G	---	---	---
H	---	---	---
I	---	---	---

ÁRBOLES Y GRAFOS

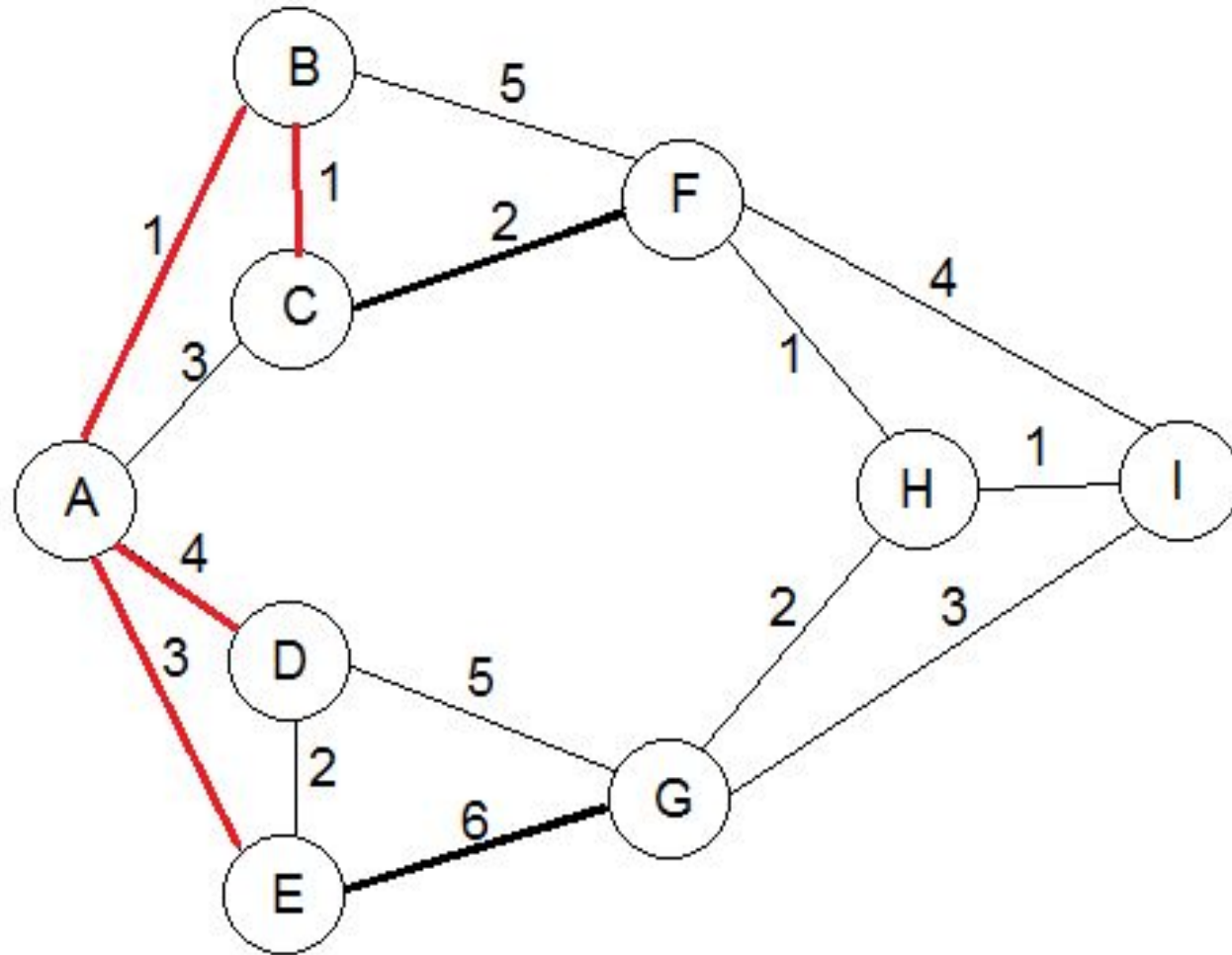
Ejemplo 2 (cont. 4):



	1 (A)	2 (B)	3 (C)	4 (E)
A	(0, A)	X	X	X
B	(1, A)	(1, A)	X	X
C	(3, A)	(2, B)	(2, B)	X
D	(4, A)	(4, A)	(4, A)	(4, A)
E	(3, A)	(3, A)	(3, A)	(3, A)
F	---	(6, B)	(4, C)	(4, C)
G	---	---	---	(9, E)
H	---	---	---	---
I	---	---	---	---

ÁRBOLES Y GRAFOS

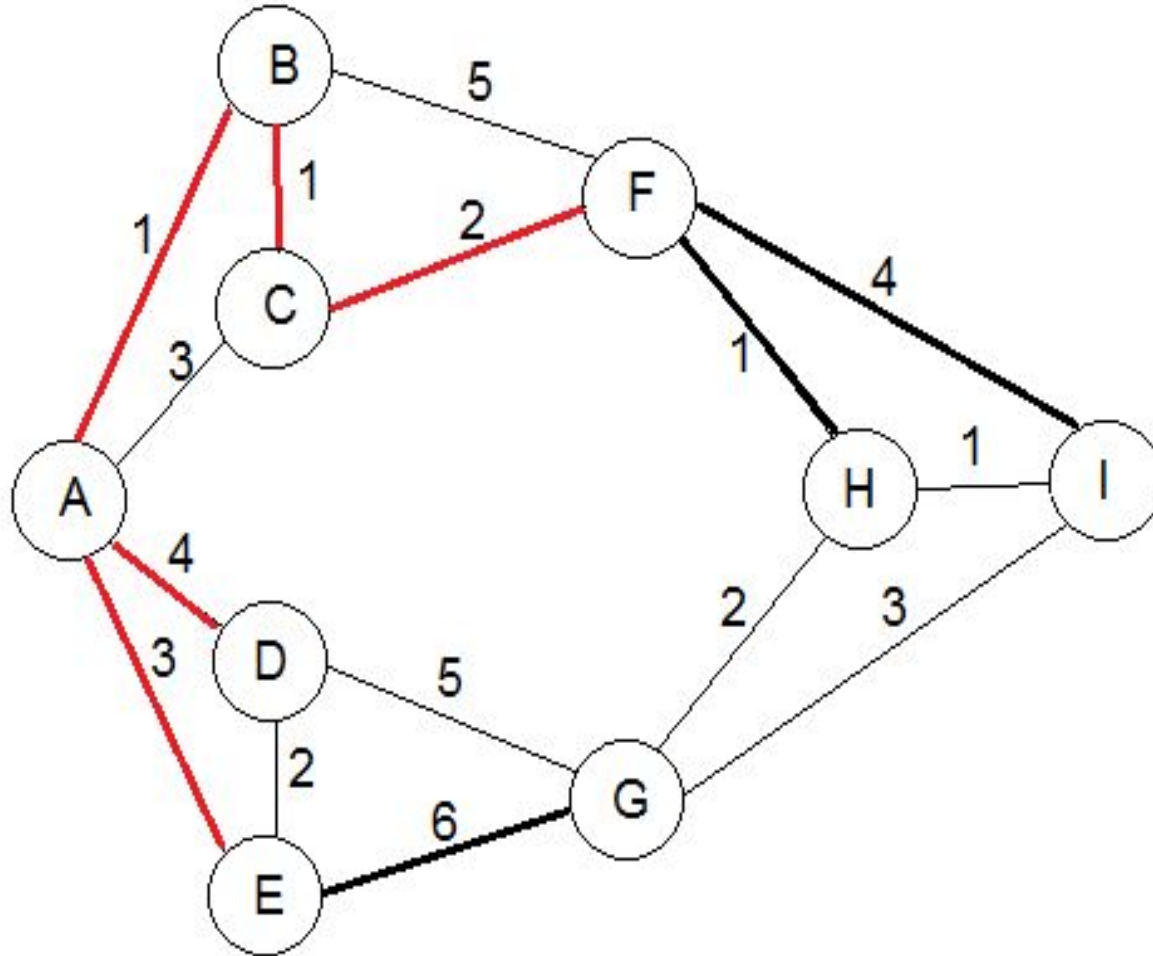
Ejemplo 2 (cont. 5):



	1 (A)	2 (B)	3 (C)	4 (E)	5 (D)
A	(0, A)	X	X	X	X
B	(1, A)	(1, A)	X	X	X
C	(3, A)	(2, B)	(2, B)	X	X
D	(4, A)	(4, A)	(4, A)	(4, A)	(4, A)
E	(3, A)	(3, A)	(3, A)	(3, A)	X
F	---	(6, B)	(4, C)	(4, C)	(4, C)
G	---	---	---	(9, E)	(9, E)
H	---	---	---	---	---
I	---	---	---	---	---

ÁRBOLES Y GRAFOS

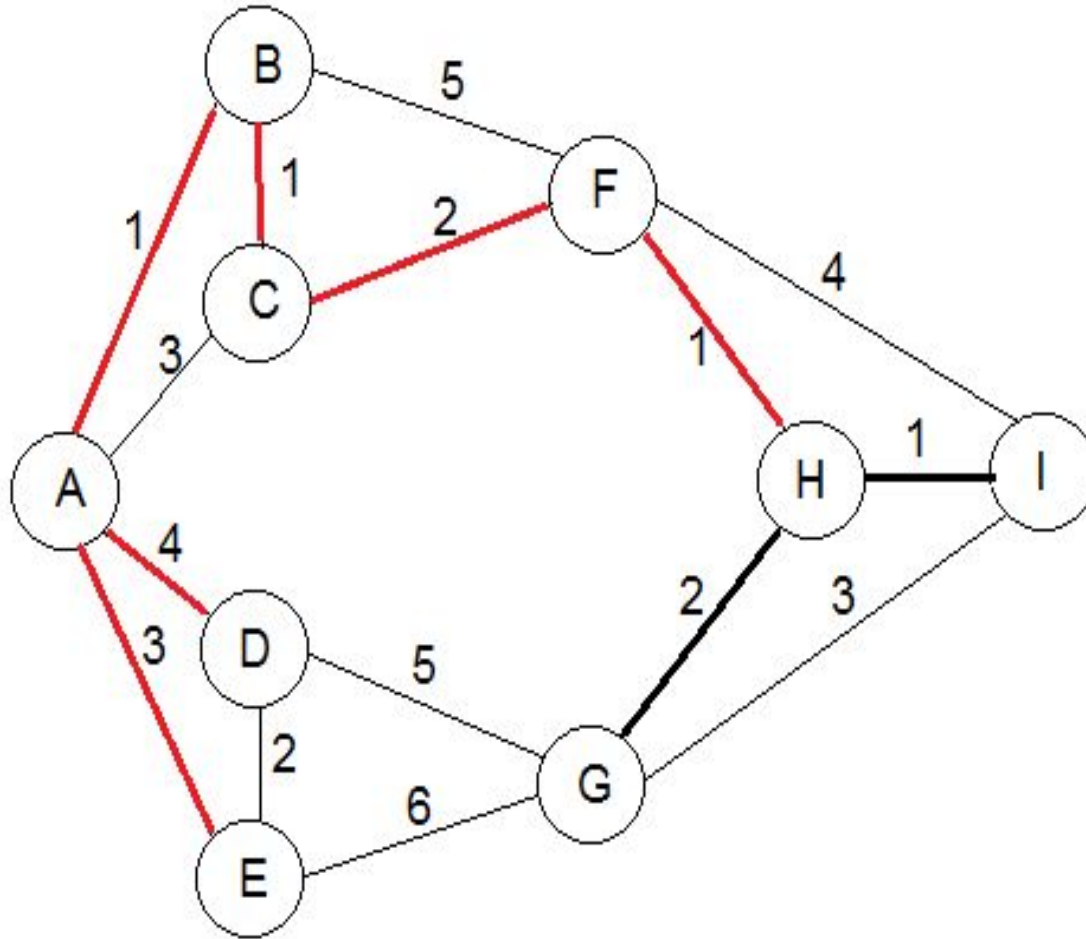
Ejemplo 2 (cont. 6):



	1 (A)	2 (B)	3 (C)	4 (E)	5 (D)	6 (F)
A	(0, A)	X	X	X	X	X
B	(1, A)	(1, A)	X	X	X	X
C	(3, A)	(2, B)	(2, B)	X	X	X
D	(4, A)	(4, A)	(4, A)	(4, A)	(4, A)	X
E	(3, A)	(3, A)	(3, A)	(3, A)	X	X
F	---	(6, B)	(4, C)	(4, C)	(4, C)	(4, C)
G	---	---	---	(9, E)	(9, E)	(9, E)
H	---	---	---	---	---	(5, F)
I	---	---	---	---	---	(8, F)

ÁRBOLES Y GRAFOS

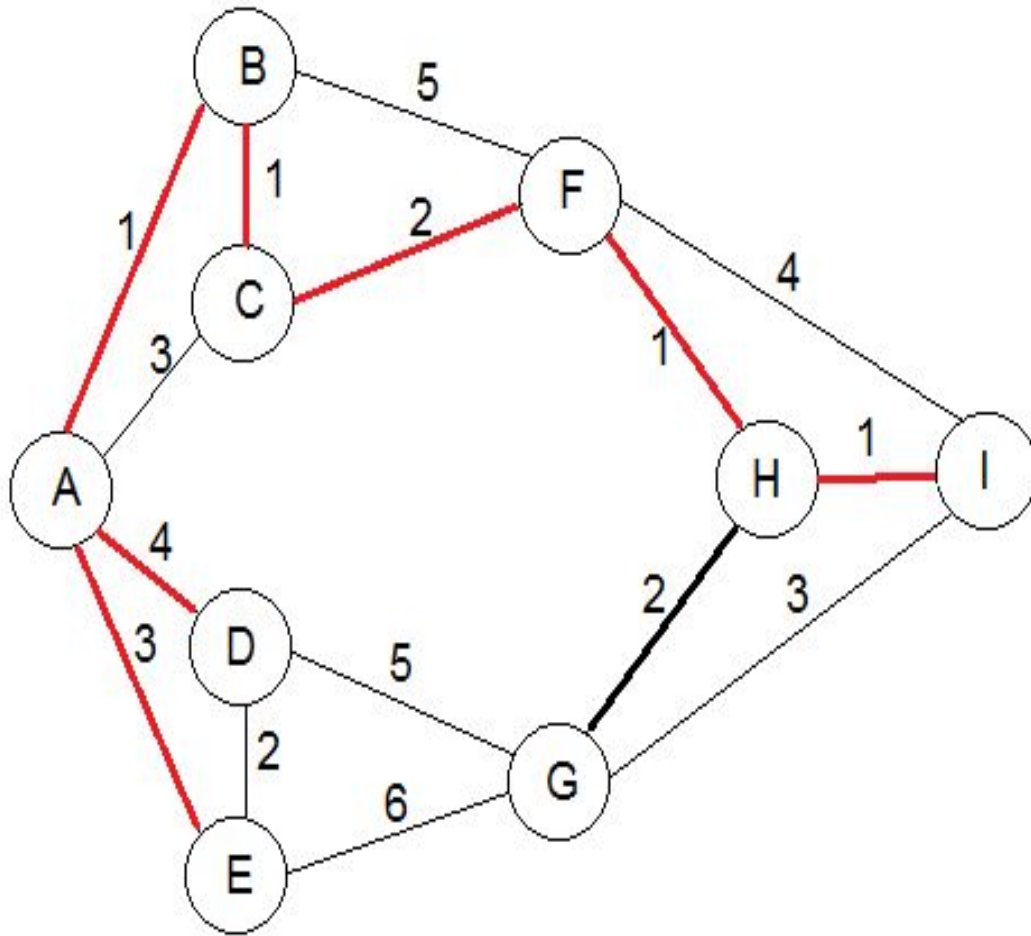
Ejemplo 2 (cont. 7):



	1 (A)	2 (B)	3 (C)	4 (E)	5 (D)	6 (F)	7 (H)
A	(0, A)	X	X	X	X	X	X
B	(1, A)	(1, A)	X	X	X	X	X
C	(3, A)	(2, B)	(2, B)	X	X	X	X
D	(4, A)	(4, A)	(4, A)	(4, A)	(4, A)	X	X
E	(3, A)	(3, A)	(3, A)	(3, A)	X	X	X
F	---	(6, B)	(4, C)	(4, C)	(4, C)	(4, C)	X
G	---	---	---	(9, E)	(9, E)	(9, E)	(7, H)
H	---	---	---	---	---	(5, F)	(5, F)
I	---	---	---	---	---	(8, F)	(6, H)

ÁRBOLES Y GRAFOS

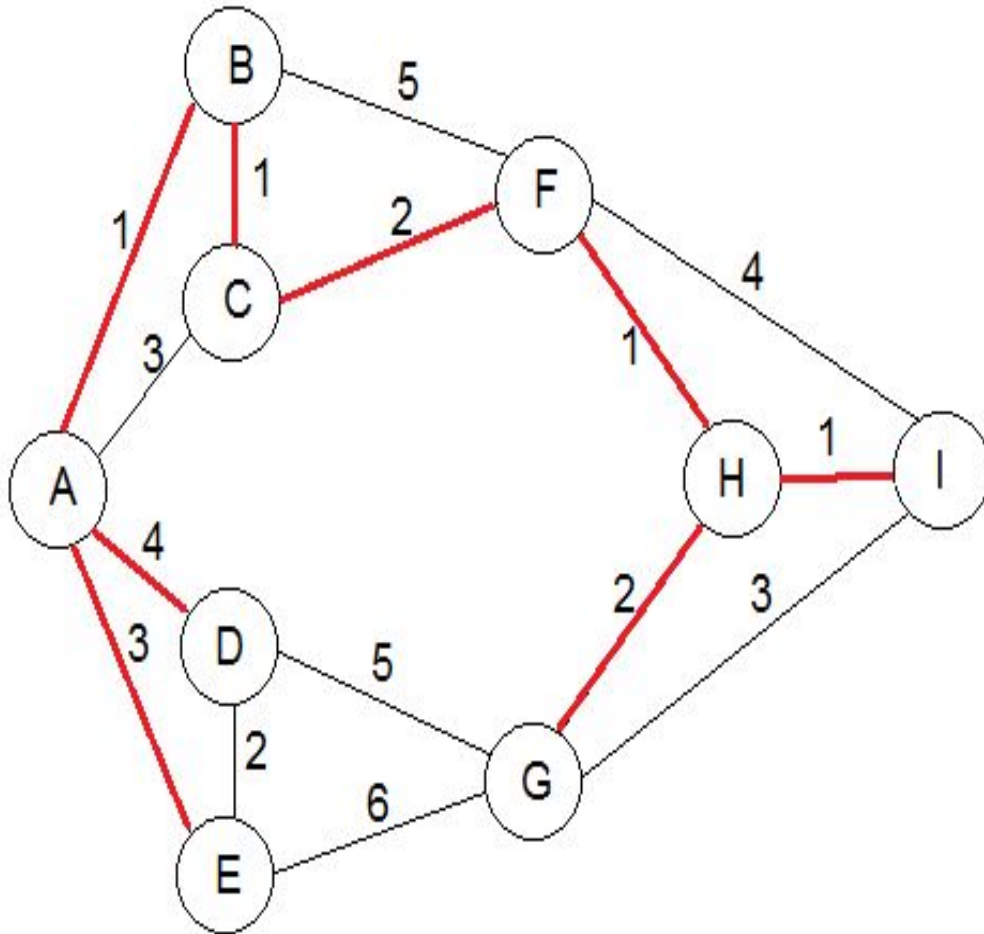
Ejemplo 2 (cont. 8):



	1 (A)	2 (B)	3 (C)	4 (E)	5 (D)	6 (F)	7 (H)	8 (I)
A	(0, A)	X	X	X	X	X	X	X
B	(1, A)	(1, A)	X	X	X	X	X	X
C	(3, A)	(2, B)	(2, B)	X	X	X	X	X
D	(4, A)	(4, A)	(4, A)	(4, A)	(4, A)	X	X	X
E	(3, A)	(3, A)	(3, A)	(3, A)	X	X	X	X
F	---	(6, B)	(4, C)	(4, C)	(4, C)	(4, C)	X	X
G	---	---	---	(9, E)	(9, E)	(9, E)	(7, H)	(7, H)
H	---	---	---	---	---	(5, F)	(5, F)	X
I	---	---	---	---	---	(8, F)	(6, H)	(6, H)

ÁRBOLES Y GRAFOS

Ejemplo 2 (cont. 9):



	1 (A)	2 (B)	3 (C)	4 (E)	5 (D)	6 (F)	7 (H)	8 (I)	9 (G)
A	(0, A)	X	X	X	X	X	X	X	X
B	(1, A)	(1, A)	X	X	X	X	X	X	X
C	(3, A)	(2, B)	(2, B)	X	X	X	X	X	X
D	(4, A)	(4, A)	(4, A)	(4, A)	(4, A)	X	X	X	X
E	(3, A)	(3, A)	(3, A)	(3, A)	X	X	X	X	X
F	---	(6, B)	(4, C)	(4, C)	(4, C)	(4, C)	X	X	X
G	---	---	---	(9, E)	(9, E)	(9, E)	(7, H)	(7, H)	(7, H)
H	---	---	---	---	---	(5, F)	(5, F)	X	X
I	---	---	---	---	---	(8, F)	(6, H)	(6, H)	X

Algoritmo A*:

Es un algoritmo que alcanza el camino mínimo entre dos nodos utilizando información del dominio. Cada nodo posee un valor denominado valor heurístico que es determinado por alguna heurística basada en información del dominio, determinando de esta manera que tan óptimo es cada nodo.

Cuando se utiliza este algoritmo, lo que se desea hallar no solamente es el camino mínimo, sino que se busca también el nodo más óptimo teniendo en cuenta la heurística y el costo del camino.

Si se busca un nodo con la mayor heurística, los costos del camino deberán restarse, si por el contrario se busca una heurística menor, los costos deberán sumarse.

Algoritmo A* (cont.):

- 1) Partiendo del nodo origen, se marca éste como nodo visitado y se agrega a una lista de nodos visitados.
- 2) Desde el nodo actual (último nodo visitado) se generan todos los nodos adyacentes al mismo y se agregan a una lista ordenada por heurística y costo del camino.
- 3) El siguiente nodo a visitar es el primero de la lista ordenada de nodos generados, el cual se elimina de la lista ordenada y se agrega a la lista de nodos visitados.
- 4) Se continúa hasta alcanzar el nodo objetivo.

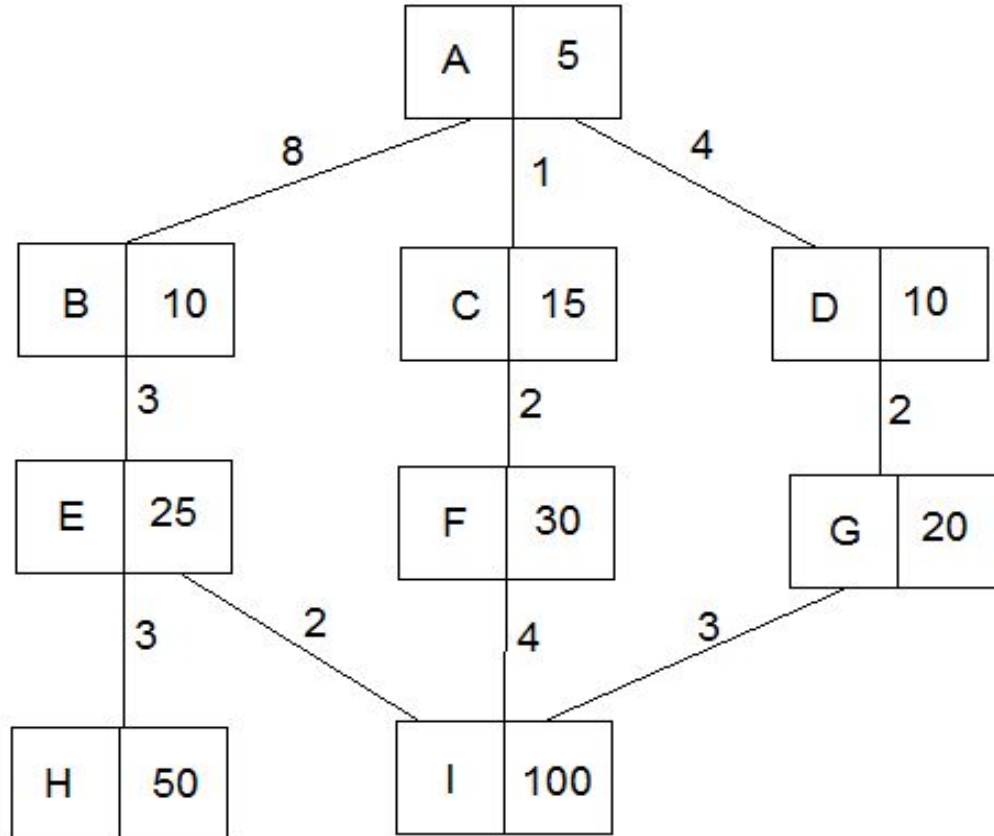
Algoritmo A* (cont. 2):

Para poder determinar el camino, una vez finalizado el algoritmo, las listas de nodos visitados y generados debe contener información adicional: el nodo desde el cual se alcanza el nodo generado.

De esta manera, se puede describir el camino encontrado revisando la lista de nodos visitados.

ÁRBOLES Y GRAFOS

Ejemplo 1: Nodo objetivo: I



Paso	Nodos visitados	Nodos generados
1	(A, A)	(C, 14, A); (D, 6, A); (B, 2, A)
2	(C, A)	(F, 27, C); (D, 6, A); (B, 2, A)
3	(F, C)	(I, 93, F); (D, 6, A); (B, 2, A)
4	(I, F)	Nodo encontrado: I

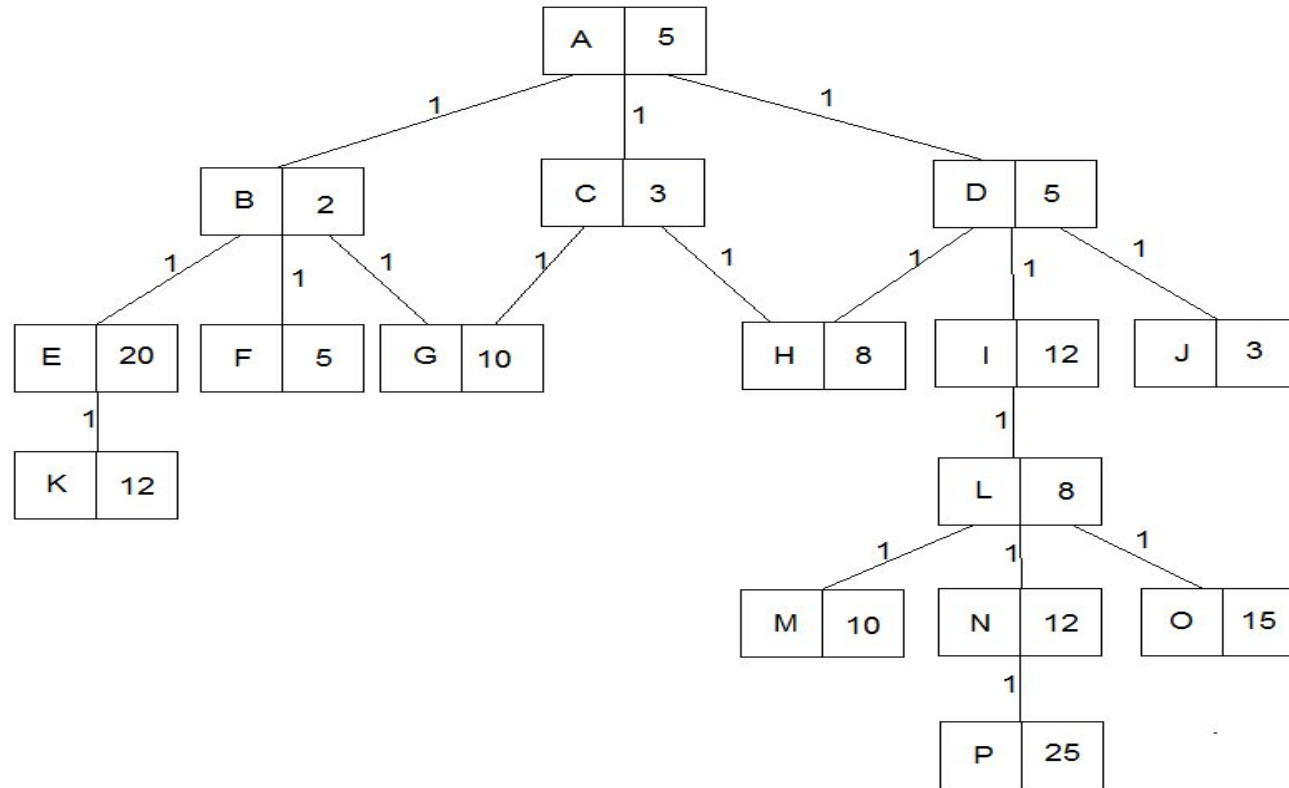
Heurística: mayor valor mejor. Nodo origen: A

Camino: (I, F)--->(F, C)--->(C, A)

Camino: A--->C--->F--->I

ÁRBOLES Y GRAFOS

Ejemplo 2: Nodo objetivo: P



Paso	Nodos visitados	Nodos generados
1	(A, A)	(D, 4, A); (C, 2, A); (B, 1, A)
2	(D, A)	(I, 10, D); (H, 6, D); (C, 2, A); (B, 1, A); (J, 1, D)
3	(I, D)	(H, 6, D); (L, 5, I); (C, 2, A); (B, 1, A); (J, 1, D)
4	(H, D)	(L, 5, I); (C, 2, A); (B, 1, A); (J, 1, D)
5	(L, I)	(O, 11, L); (N, 8, L); (M, 6, L); (C, 2, A); (B, 1, A); (J, 1, D)
6	(O, L)	(N, 8, L); (M, 6, L); (C, 2, A); (B, 1, A); (J, 1, D)
7	(N, L)	(P, 20, N); (M, 6, L); (C, 2, A); (B, 1, A); (J, 1, D)
8	(P, N)	Nodo encontrado: P

Heurística: mayor valor mejor. Nodo origen: A
Camino: (P, N) ----> (N, L) ----> (L, I) ----> (I, D) ----> (D, A)
Camino: A ----> D ----> I ----> L ----> N ----> P

ÁRBOLES Y GRAFOS

Beam Search:

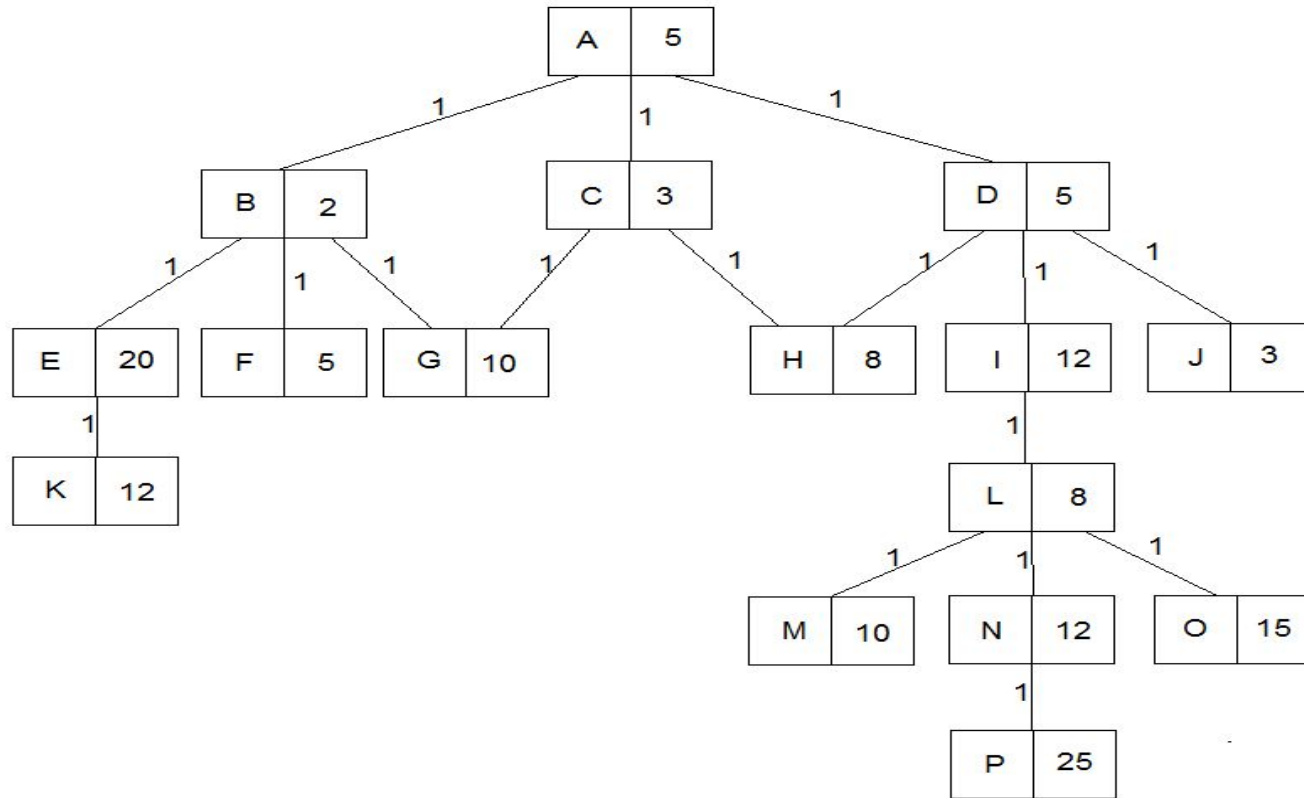
Es una modificación del Algoritmo A* que busca economizar memoria.

Funciona como el Algoritmo A* pero con una restricción: el número (tamaño) de nodos de la lista de nodos generados es limitada. Supone que entre los primeros nodos estará el nodo que lleva al mejor camino.

Cuando no sucede esto, el Algoritmo cae en óptimos locales y no llega a encontrar el nodo buscado.

ÁRBOLES Y GRAFOS

Ejemplo: Nodo objetivo: P (N=2)



Paso	Nodos visitados	Nodos generados
1	(A, A)	(D, 4, A); (C, 2, A)
2	(D, A)	(I, 10, D); (H, 6, D)
3	(I, D)	(H, 6, D); (L, 5, I)
4	(H, D)	(L, 5, I)
5	(L, I)	(O, 11, L); (N, 8, L)
6	(O, L)	(N, 8, L)
7	(N, L)	(P, 20, N)
8	(P, N)	Nodo encontrado: P

Heurística: mayor valor mejor. Nodo origen: A
Camino: (P, N) ---> (N, L) ---> (L, I) ---> (I, D) ---> (D, A)
Camino: A ---> D ---> I ---> L ---> N ---> P

ÁRBOLES Y GRAFOS

Otros Métodos con Información del Dominio:

Escalada Simple: genera en orden los nodos hasta encontrar uno mejor, y lo visita. No tiene vuelta atrás, por lo que puede caer en óptimos locales.

Escalada Máxima Pendiente: supone una mejora con el método de Escalada Simple. Genera todos los hijos y visita el mejor. Tampoco tiene vuelta atrás, por lo que también puede caer en óptimos locales.